SportsOS: An AI-Powered Sports ERP for IIT (ISM) Dhanbad

Thesis Mid-Evaluation Report

submitted in partial fulfilment of the requirements for the degree of

Integrated Master of Technology

in

Mathematics and Computing

Submitted by

Anup Kumar

Admission No.: 22JE0143

Under the guidance of

Prof. Scindhiya Laxmi

Department of Mathematics and Computing

Indian Institute of Technology (Indian School of Mines), Dhanbad

Dhanbad, Jharkhand, India

13 November 2026

ABSTRACT

Sports administration at IIT (ISM) Dhanbad depends on paper registers, spreadsheets and informal messaging, which leaves facility usage, attendance and equipment stock without real-time visibility or a reliable audit trail. This thesis designs and develops SportsOS, a role-aware sports Enterprise Resource Planning system that unifies facility booking, verified attendance, equipment inventory, tournaments, complaints, announcements and analytics for the institute's sports ecosystem.

The work is positioned in Software Engineering and Information Systems. Requirements were derived from the approved proposal and modelled as 18 stakeholder roles, 70 permissions and four scope types. The system is built as a TypeScript monorepo with a Next.js 15 web client, an Expo SDK 54 mobile client, and a NestJS 11 REST API over a 31-entity PostgreSQL schema managed with Prisma. Its main engineering contributions are: scope-aware, time-bounded role-based access control with two-person approval for privileged roles; proof-of-presence attendance combining time-based one-time passwords, GPS geofencing and device binding; database-enforced consistency for bookings and stock; and a SHA-256 hash-chained audit ledger. An optional retrieval-augmented assistant operates on permission-filtered aggregate data.

For this mid-evaluation, the author re-executed the full verification suite on 12 November 2026: 71 of 71 unit tests, 83 of 83 API smoke checks, 200 of 200 role-based access-control and workflow checks across all 18 roles, 21 of 21 browser checks, and a set of targeted checks run by the author all passed. The evaluation also confirms the limitations: there is no password-reset flow and no shared cache; the mobile client has been type-checked but not device-tested; the AI layer has been verified only in offline mode; all data is synthetic; and load, penetration and user testing remain to be done. The evaluation is therefore preliminary, and the remaining months are planned for stakeholder validation and hardening.

Keywords: sports management information system, enterprise resource planning, role-based access control, verified attendance, geofencing, web application security, NestJS, Next.js, PostgreSQL.

TABLE OF CONTENTS

Abstract ..... **i**

List of Figures ..... **v**

List of Tables ..... **vi**

List of Abbreviations ..... **vii**

CHAPTER 1: INTRODUCTION **1**

1.1 Introduction 1
1.2 Institutional Background 1
1.3 Sports Ecosystem at IIT (ISM) 2
1.4 Existing Operational Context 2
1.5 Problem Statement 3
1.6 Motivation 3
1.7 Research Objectives 4
1.8 Project Scope 4
1.9 Expected Contributions 5
1.10 Report Organisation 5

CHAPTER 2: LITERATURE REVIEW AND RELATED WORK **6**

2.1 Introduction 6
2.2 Digital Transformation in Sports Administration 6
2.3 Sports Management Information Systems 6
2.4 Enterprise Information Systems and ERP 7
2.5 Integrated Facility Management 7
2.6 Multi-Stakeholder Information Systems 7
2.7 Role-Based Access Control 8
2.8 Web Application Authentication and Security 8
2.9 Related Sports Management Applications 9
2.10 Comparative Analysis 9
2.11 Research Gap 10

CHAPTER 3: REQUIREMENTS ENGINEERING AND SYSTEM ANALYSIS **11**

3.1 Introduction 11
3.2 Stakeholder Identification 11
3.3 Stakeholder Hierarchy 11
3.4 User Roles and Responsibilities 12
3.5 Functional Requirements 12
3.6 Non-Functional Requirements 13
3.7 Use Case Analysis 14
3.8 User Stories and Workflows 14
3.9 System Constraints 15
3.10 Development Methodology 15

CHAPTER 4: SYSTEM ARCHITECTURE AND DATABASE DESIGN **17**

4.1 System Overview 17
4.2 Architectural Approach 17
4.3 Frontend Architecture 18

4.4 Backend Architecture....18
4.5 API Design....19
4.6 Database Architecture....19
4.7 Entity Relationship Diagram....20
4.8 Authentication Architecture....21
4.9 Role-Based Access Control Design....22
4.10 Data Flow Diagrams....23
4.11 Security and Privacy Considerations....24
4.12 Infrastructure and Deployment Architecture....25
CHAPTER 5: IMPLEMENTATION AND DEVELOPMENT....26
5.1 Development Environment....26
5.2 Technology Stack and Justification....26
5.3 Monorepo Structure....27
5.4 Frontend Implementation....27
5.5 Backend Implementation....28
5.6 Database Implementation....28
5.7 Authentication and User Management....29
5.8 Role-Based Access Control....29
5.9 Sports Club Management....29
5.10 Facility and Ground Booking....30
5.11 Attendance Management....31
5.12 Equipment and Inventory....32
5.13 Tournament and Event Management....33
5.14 Complaint Management....34
5.15 Notifications and Announcements....35
5.16 Dashboard and Analytics....35
5.17 Mobile Application....35
5.18 AI-Assisted Features and Future Integration....36
5.19 Module-Wise Implementation Status....36
CHAPTER 6: SOFTWARE TESTING AND EVALUATION....38
6.1 Introduction....38
6.2 Testing Strategy....38
6.3 Unit Testing....39
6.4 Integration Testing....39
6.5 Functional Testing....40
6.6 Authentication Testing....40
6.7 Authorisation and RBAC Testing....41
6.8 Database Testing....41
6.9 API Testing....41
6.10 End-to-End Testing....41
6.11 Security Testing....42
6.12 Preliminary Evaluation....42
CHAPTER 7: RESULTS, LIMITATIONS AND FUTURE WORK....44
7.1 Current Development Status....44
7.2 Achievement Against Research Objectives....44

7.3 Preliminary System Demonstration....44
7.4 Engineering Contributions....45
7.5 Current Limitations....45
7.6 Technical Challenges....46
7.7 Future Enhancements....47
7.8 Revised Development Roadmap....47
7.9 Conclusion....48
REFERENCES....49
APPENDIX A: REPOSITORY STRUCTURE....52
APPENDIX B: SELECTED API DOCUMENTATION....53
APPENDIX C: DATABASE ENTITY CATALOGUE....55
APPENDIX D: IMPLEMENTATION EVIDENCE MATRIX....56
APPENDIX E: ADDITIONAL SCREENSHOTS....57
APPENDIX F: DETAILED TEST CASES....63

LIST OF FIGURES

Figure 3.1: Use case diagram of principal SportsOS actors and use cases (derived from implemented routes)..... 14

Figure 4.1: System architecture of SportsOS (all versions verified from package manifests) 17

Figure 4.2: Entity relationship diagram of core SportsOS entities (crow's-foot notation, from schema.prisma)..... 21

Figure 4.3: Authentication sequence (login and subsequent authenticated request)..... 22

Figure 4.4: RBAC enforcement pipeline from request to handler..... 23

Figure 4.5: Level-1 data flow diagram..... 24

Figure 4.6: Deployment architecture as configured in docker-compose.yml..... 25

Figure 5.1: Sports Officer dashboard with KPIs and the approval queue (synthetic demo data)..... 27

Figure 5.2: Facility booking workflow (activity diagram from bookings.service.ts)..... 30

Figure 5.3: Attendance check-in validation workflow (order as implemented in attendance.service.ts)..... 31

Figure 5.4: Coach console with a live verified-attendance session (created during evaluation) 32

Figure 5.5: Equipment issue and return workflow..... 33

Figure 5.6: Knock-out bracket with seeded byes and automatic advancement (synthetic tournament)..... 34

Figure 5.7: Complaint lifecycle state machine (complaints.service.ts)..... 34

Figure 7.1: Revised development roadmap (completed work and planned activities)..... 47

Figure E.1: Administrator view of roles and the editable role × permission matrix (synthetic data)..... 57

Figure E.2: Facility page with the daily availability grid (Main Football Ground) (synthetic data)..... 57

Figure E.3: Inventory view with stock levels and low-stock alerts (synthetic data)..... 58

Figure E.4: Analytics page: participation, utilisation and forecast views (synthetic data)…. 58

Figure E.5: SportsOS AI assistant page (offline retrieval mode) (synthetic data)..... 59

Figure E.6: Sign-in page (synthetic data)..... 59

Figure E.7: Student dashboard (synthetic data)..... 60

Figure E.8: Booking approvals for the Sports Officer (synthetic data)..... 60

Figure E.9: Clubs as seen by a coach (synthetic data)..... 61

Figure E.10: Complaints page for a student (synthetic data)..... 61

Figure E.11: Hash-chained audit ledger view (synthetic data)..... 62

Figure E.12: User administration (synthetic data)..... 62

LIST OF TABLES

Table 1.1: Research objectives from the approved proposal....4

Table 2.1: Comparison of reviewed work with SportsOS (claims limited to what each source reports)....9

Table 3.1: Implemented roles and default permission counts (measured)....12

Table 3.2: Functional requirements and current status....12

Table 3.3: Non-functional requirements (ISO/IEC 25010 characteristics)....13

Table 3.4: Representative user stories with acceptance criteria....15

Table 4.1: Security controls mapped to ASVS 5.0.0 areas (indicative; not an ASVS assessment)....24

Table 5.1: Technology stack: versions verified from package manifests, with the role each plays in SportsOS....26

Table 5.2: Module-wise implementation status at mid-evaluation....36

Table 6.1: Test levels, tools and results of the 12 November 2026 run....38

Table 6.2: Unit test suites (Jest), 71 tests, all passing....39

Table 6.3: Representative test cases (complete tables in Appendix F)....40

Table 7.1: Achievement against the proposal's objectives....44

Table 7.2: Known limitations at mid-evaluation....45

Table A.1: Source size by area (lines, measured with wc -l on .ts/.tsx/.mjs/.cjs/.prisma/.sql files)....52

Table B.1: API surface by module (counted from controller decorators)....53

Table B.2: Selected endpoints....53

Table C.1: All 31 Prisma models grouped by domain....55

Table C.2: Seeded synthetic data volumes (run of 12 Nov 2026)....55

Table D.1: Requirement → implementation → verification evidence....56

Table F.1: Functional test cases: booking, inventory and complaints (results from the smoke and e2e runs)....63

Table F.2: Authentication test cases....63

Table F.3: Authorisation and RBAC test cases....64

Table F.4: Database test cases (author's checks)....64

Table F.5: Attendance verification test cases (author's run against a live session)....65

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
ASVS	Application Security Verification Standard (OWASP)
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
DFD	Data Flow Diagram
DTO	Data Transfer Object
e2e	End-to-end (testing)
ERD	Entity Relationship Diagram
ERP	Enterprise Resource Planning
GPS	Global Positioning System
HMAC	Hash-based Message Authentication Code
HOTP	HMAC-based One-Time Password
IST	Indian Standard Time
JWT	JSON Web Token
LLM	Large Language Model
MIS	Management Information System
NAT	Network Address Translation
ORM	Object-Relational Mapping
OTP	One-Time Password
OWASP	Open Worldwide Application Security Project
PESO	Sports Officer (abbreviation as used in the proposal)
QR	Quick Response (code)
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPEC	Sports & Physical Education Center
SSE	Server-Sent Events
SW	Students' Welfare
TOTP	Time-based One-Time Password
UAT	User Acceptance Testing

CHAPTER 1

INTRODUCTION

1.1 Introduction

Institutional sport is an information-intensive activity. Every practice session, ground allocation, equipment loan, tournament fixture and complaint produces records that several offices must create, approve, reconcile and report on. When those records live in paper registers, personal spreadsheets and messaging groups, the institution loses the ability to see its own operations in real time, to audit them afterwards, and to plan from evidence. This thesis addresses that problem for IIT (ISM) Dhanbad through **SportsOS**, a role-aware sports Enterprise Resource Planning (ERP) system delivered as a web application, a mobile application and a REST API over a single relational database.

The thesis is positioned in Software Engineering and Information Systems rather than in machine learning. Its contribution is the requirements analysis, architecture, implementation and verification of an integrated multi-stakeholder information system. Artificial-intelligence features are treated as an optional capability layered on top of the transactional core, and they are reported only to the extent that they have been verified.

This mid-evaluation report documents the state of the project on 13 November 2026, three months into the nine-month plan of the approved proposal [1]. It reports what has been designed, implemented, integrated and tested, and it separates those results from what remains proposed or planned. All test results quoted in Chapter 6 were re-executed by the author on 12 November 2026 against a freshly migrated and seeded PostgreSQL 16 database; no figure in this report is carried over from earlier documentation without being re-run.

1.2 Institutional Background

IIT (ISM) Dhanbad is a residential technical institute whose student life is organised around hostels, clubs and institute-level teams. Sport is administered under the Dean (Students' Welfare) through the Sports & Physical Education Center (SPEC). The SPEC web pages hosted on the institute's people server describe SPEC's remit as including the organisation of intramural competitions such as the **Annual Sports Meet** and the **Inter-Hostel General Championship** [2].

The proposal [1] uses the term "Sports and Physical Education Cell", while SPEC's own pages use "Sports & Physical Education Center" [2]. This report follows the institute's usage. The two institute pages named in the evaluation brief, the SPEC page and the Dean (Students' Welfare) facilities page [3], [4], could not be reached from the isolated evaluation environment, because the host was outside its network allow-list. Institutional statements in this report are therefore limited to what could be confirmed from the SPEC activity and facility pages [2], [5]. Those pages are undated, so their content should be re-confirmed with SPEC before final submission.

1.3 Sports Ecosystem at IIT (ISM)

According to the SPEC facilities page [5], the campus sports infrastructure includes the following:

- a **Lower Ground** with a standard-size football ground and an eight-lane, 400 m mud track with floodlights;
- a **Sports Complex** with floodlit basketball, volleyball and tennis courts;
- a **Students Activity Centre** with a gymnasium for boys and girls, four synthetic-floor badminton courts, a yoga centre, a boxing ring and a billiards room;
- a 50 m standard-size **swimming pool**.

These facilities serve three kinds of participation: competitive (institute teams, the Annual Sports Meet and inter-institute events), inter-hostel (the Inter-Hostel General Championship), and recreational (casual use by any student). SportsOS models all three. A facility can be booked casually, reserved for club practice, or blocked for an event or maintenance. Tournaments are organised around hostel teams, and attendance is verified per practice session.

The official pages consulted do not list individual coaches, ground staff or their reporting lines, and none were available to the author in verifiable form. This report therefore names **no** sports personnel. The stakeholder roles used in the system (Chapter 3) derive from the approved proposal [1]. They are a requirements model to be validated with SPEC, not an official organisational chart.

1.4 Existing Operational Context

The proposal [1] records five operational areas: facility allocation and scheduling, student participation and attendance, tournament and event management, equipment issue and

return, and administrative coordination. In each of them, work is currently carried out through paper registers, spreadsheets and informal messaging groups. This observation comes from the author's proposal and has not yet been measured. A structured requirements-validation exercise with SPEC staff is scheduled for the next phase (Section 7.8). Until that exercise is done, this report treats the baseline as a stated premise rather than as measured data.

1.5 Problem Statement

Sports operations at IIT (ISM) Dhanbad are fragmented across offline channels. As a result, administrators lack reliable real-time visibility of facility usage, attendance and equipment stock. The records that do exist are easy to manipulate (for example, proxy entries in paper attendance registers) and hard to audit. The institute needs an integrated, role-aware web and mobile ERP that turns these operations into auditable and actionable data while giving each stakeholder access only to the functions and records that their role requires [1].

In software-engineering terms, the problem decomposes into four questions that this thesis answers in turn:

1. Which stakeholders, roles and permissions must the system represent, and how can authority be scoped (to a club, hostel, sport or facility) and bounded in time without hard-coding the organisation chart?
2. Which architecture and data model can unify bookings, attendance, inventory, events and complaints so that concurrent operations stay consistent?
3. How can attendance and equipment records be made tamper-resistant and auditable in practice?
4. How can the implementation be verified systematically, especially its authorisation behaviour across all roles?

1.6 Motivation

Three considerations motivate the work. First, sports digitalisation research has concentrated on single solutions for single stakeholders, and recent reviews call for work that considers multiple stakeholders together [6]. A campus sports ERP is such a setting, with students, coaches, staff, student office-bearers and leadership sharing one system. Second, integrated sports-facility information systems have been associated with better user experience and service quality in a recent systematic review [7]. However, that review

found few studies, and none in the Indian institutional context. Third, access control is the dominant risk in multi-role systems: OWASP ranks broken access control as the leading web-application risk category [8], and ASVS 5.0.0 dedicates verification requirements to it [9]. A sports ERP with eighteen roles is a realistic, non-trivial test bed for rigorous role-based access control (RBAC).

1.7 Research Objectives

The six objectives of the approved proposal [1] are retained unchanged and are tracked throughout this report:

Table 1.1: *Research objectives from the approved proposal*

ID	Objective	Measured in this report by
O1	Centralise sports operations: integrate facility booking, tournaments, attendance, inventory and events in one platform.	Single schema and API covering all modules (Ch. 4–5)
O2	Automate administrative processes: scheduling, approvals, record management and reporting.	Approval workflows, scheduled jobs, notifications (Ch. 5)
O3	Implement role-based access control for students, administrators, coaches, SPEC, the Sports Council and others.	RBAC design and the 18-role enforcement tests (Ch. 4, 6)
O4	Enable real-time analytics and reporting.	Analytics endpoints and dashboards (Ch. 5)
O5	Enhance the student experience through one portal.	Student-facing web and mobile functions (Ch. 5)
O6	Build a scalable, secure, future-ready system that can support AI features.	Architecture, security controls, AI layer and their limits (Ch. 4–7)

1.8 Project Scope

In scope for the thesis are the following: authentication and user management; scoped, time-bound RBAC; facilities and booking; clubs and rosters; OTP- and GPS-verified attendance; equipment inventory with QR codes; tournaments and fixtures; complaints; notifications and announcements; operational tasks; dashboards and analytics; a mobile client for on-ground use; an AI-assisted question-answering and reporting layer; and an import path for institutional reference data.

Out of scope are payment processing, integration with the institute's central ERP or single sign-on (which would require institute authorisation), biometric attendance, and any processing of real student data before institutional approval. Every record in the current system is synthetic and labelled as such (Section 5.6).

1.9 Expected Contributions

- A requirements model of a campus sports ecosystem with 18 stakeholder roles, 70 permissions and four scope types (Chapter 3).
- A reference architecture and a 31-entity relational schema for a sports ERP, including concurrency-safe booking and inventory and a hash-chained audit ledger (Chapter 4).
- A working prototype with web, mobile and API tiers (Chapter 5).
- A verification approach for multi-role systems that probes every protected endpoint class with every role, plus browser-level checks per role (Chapter 6).

1.10 Report Organisation

Chapter 2 reviews related work and states the research gap. Chapter 3 presents requirements engineering and system analysis. Chapter 4 describes the architecture and database design. Chapter 5 reports the implementation module by module. Chapter 6 presents the testing strategy and the results actually obtained. Chapter 7 assesses progress against the objectives, lists limitations and presents the revised roadmap. The appendices contain the repository structure, selected API documentation, the full entity list, the implementation evidence matrix and additional screenshots.

CHAPTER 2

LITERATURE REVIEW AND RELATED WORK

2.1 Introduction

This chapter reviews five strands of literature relevant to SportsOS: digital transformation in sport, sports management information systems, enterprise systems, multi-stakeholder systems, and access control and web-application security. It closes with a comparative analysis and the research gap. Bibliographic details were checked against publisher or indexing records. Where only abstract-level information could be confirmed, the review limits its claims to that level.

2.2 Digital Transformation in Sports Administration

Vial defines digital transformation as a process in which digital technologies trigger strategic responses that change how an organisation creates value, and stresses that structural and cultural changes accompany the technology [10]. For a sports administration, this implies that digitising a register is not enough: the workflows built around the register (approvals, escalations, reporting) must also be redesigned.

Haffner, Oshri and Kotlarsky review information-systems research on sports digitalisation through a stakeholder lens [6]. They find that existing studies typically analyse the effect of one digital solution on one stakeholder group, and they propose directions for research that considers the interplay of multiple stakeholders. SportsOS takes this multi-stakeholder view as a design premise: one transaction, such as a booking, simultaneously involves a requesting student, an approving officer, security staff at the gate and leadership reading utilisation analytics.

2.3 Sports Management Information Systems

Saputra and Asmawati systematically review integrated management information systems for sports facilities, covering seven articles from 2018 to 2023 [7]. They identify inventory management, facility reservations, cleanliness supervision and performance reporting as the components of an integrated approach. They associate such integration with improved user experience and service quality, through access to information, online transactions and

feedback channels. The small size of the reviewed corpus is itself a finding: integrated sports-facility systems remain thinly studied.

Shahrom and Mohamad Sukri present a swimming-facility management system for the sport centre of Universiti Tun Hussein Onn Malaysia [11]. The system supports bookings and payments for students, staff and visitors, and it implements role-based access control with two-factor authentication (a strong password plus a one-time password). It is close to SportsOS in setting (a university sport centre) and in its concern with authentication. Its scope, however, is a single facility type, and its OTP serves login rather than proof of physical presence.

2.4 Enterprise Information Systems and ERP

Davenport characterised enterprise systems as integrated packages whose main promise, and main risk, is the integration of an organisation's information flows into a single database [12]. Klaus, Rosemann and Gable describe ERP as standard software that integrates business processes and data across functions [13]. Two properties from this literature carry over to SportsOS. The first is a **single shared data model**, so that a facility's bookings, attendance sessions, tournaments and complaints refer to one `Facility` record. The second is **process orientation**, so that an approval, rejection or escalation is a recorded state transition rather than an informal message. SportsOS is not a configurable commercial ERP. It is a domain-specific system built on the same integration principle.

2.5 Integrated Facility Management

Facility management combines scheduling (who may use which space, and when), condition management (maintenance status, closures) and asset management (equipment stock, damage and loss). The integrated systems reviewed in [7] place reservations and inventory together for this reason. The engineering problem they share is **consistency under concurrency**. Two students must not both obtain the last slot or the last ball, and stock counters must never become negative. This requirement led to the transactional design in Section 4.6, which uses database advisory locks and conditional updates [14].

2.6 Multi-Stakeholder Information Systems

Stakeholder theory holds that an organisation must account for every group that affects or is affected by its objectives [15]. In an information system, stakeholders translate into roles

with different information needs and different authority. Two practical tensions arise. The first is that **seniority does not imply operational authority**: a Director needs dashboards but should not approve individual bookings. The second is that **student office-bearers act in limited scopes**: a hostel sports secretary should register teams only for that hostel. The SportsOS role model encodes both tensions explicitly (Section 4.9).

2.7 Role-Based Access Control

The NIST RBAC model of Sandhu, Ferraiolo and Kuhn organises access control into four levels: flat, hierarchical, constrained and symmetric RBAC [16]. It was later formalised as a proposed NIST standard [17]. In core RBAC, users are assigned to roles, permissions are assigned to roles, and a user's session activates a subset of those roles. Hierarchical RBAC adds inheritance between roles. Constrained RBAC adds separation-of-duty rules.

SportsOS implements core RBAC with sessions, using an explicit active-role selection that must be validated. It deliberately does **not** use role inheritance: each role's permissions are listed explicitly, so organisational seniority confers no implicit rights. It adds two refinements that the base model leaves to the application. The first is **scoped assignments** (a role held only for a club, hostel, sport or facility). The second is **time-bounded assignments**, for temporary volunteers. Separation of duty is applied to specific workflows, for example the rule that the author of an announcement or event proposal cannot approve it, and two-person approval for privileged roles.

2.8 Web Application Authentication and Security

OWASP ASVS 5.0.0, released in May 2025, provides verification requirements for web applications across authentication, session management, authorisation, validation and other areas [9]. The OWASP Top 10 (2021) lists broken access control first [8]. The relevant SportsOS mechanisms are the following:

- passwords stored as bcrypt hashes [18];
- stateless bearer tokens following the JSON Web Token standard [19];
- per-account rate limiting;
- strict input whitelisting;
- standard security headers;
- time-based one-time passwords (TOTP) [20], [21] used not for login but as a proof-of-presence factor in attendance.

Section 4.11 maps these controls to the ASVS areas and records the gaps that remain, such as browser token storage and the absence of a password-reset flow.

2.9 Related Sports Management Applications

The applications described in the reviewed literature fall into three groups:

1. **Booking-centric facility systems**, such as the single-facility system in [11], which digitise reservations (and sometimes payments) for one venue type.
2. **Integrated facility MIS** of the kind surveyed in [7], which combine reservations, inventory and reporting for one sports centre.
3. **Analytics- and fan-oriented sports digitalisation**, which dominates the literature reviewed in [6] and targets professional sport rather than campus administration.

No reviewed system combines verified physical attendance, scoped multi-role governance of student office-bearers, inventory with a reconciled ledger, and tournament management in an academic-institution setting.

2.10 Comparative Analysis

Table 2.1: *Comparison of reviewed work with SportsOS (claims limited to what each source reports)*

Work	Setting	Booking	Attendance verification	Inventory	Events	Access control	Audit trail
Haffner et al. 2025 [6]	IS literature review, sports digitalisation	—	—	—	—	Stakeholder lens (conceptual)	—
Saputra & Asmawati 2025 [7]	Systematic review, sports facilities	Reported	Not reported	Reported	Not reported	Not reported	Not reported
Shahrom & Mohamad Sukri 2025 [11]	University pool facility	Yes (+ payments)	No (OTP for login)	Not reported	Not reported	RBAC + 2FA	Not reported
NIST RBAC 2000 [16]	Access-control model	—	—	—	—	Flat → symmetric RBAC	—
SportsOS (this work)	Institute-wide campus sports ERP	Yes, concurrency-safe	TOTP + GPS geofence + device binding	Ledger + QR	Knock-out & round-robin	Scoped, time-bound RBAC; 18 roles × 70 permissions	SHA-256 hash chain

2.11 Research Gap

The literature supplies the principles (enterprise integration, stakeholder awareness, RBAC and verification standards) but not an integrated, verifiable system for an academic sports ecosystem in which:

1. student office-bearers hold authority only within a hostel, sport or club;
2. attendance must be provably physical rather than self-declared;
3. equipment and booking records must stay consistent under concurrent use and be auditable afterwards;
4. authorisation behaviour is tested systematically across every role.

SportsOS addresses this institution-specific gap. Its research value lies in the requirements model, the architecture and the verification method, which other institutes could reuse, rather than in any single algorithm.

CHAPTER 3

REQUIREMENTS ENGINEERING AND SYSTEM ANALYSIS

3.1 Introduction

Requirements were derived from the approved proposal [1] and refined during design. They are written in the style recommended by ISO/IEC/IEEE 29148 (necessary, unambiguous, verifiable) [22], and the non-functional requirements are classified using the ISO/IEC 25010 quality model [23]. Requirement IDs are traced to implementation and tests in Appendix D. Validation with SPEC stakeholders has **not yet** taken place and is planned (Section 7.8).

3.2 Stakeholder Identification

Stakeholders were identified from the governance structure in the proposal. Each stakeholder became one role in the system, giving 17 organisational roles plus a System Administrator role for platform operations. The classification follows the proposal's grouping: leadership; sports administration and staff; student council; event administration; the student access layer; and the platform.

3.3 Stakeholder Hierarchy

The proposal models the Director and the Dean (Students' Welfare) at the top, then the President of SPEC. Reporting to SPEC are the Sports Officer (PESO), the General Secretary and the Event Committee. The Sports Officer oversees coaches, ground staff, the equipment manager, security staff and operations staff. The General Secretary coordinates the Sports Council, sport coordinators and hostel sports secretaries. Students sit in a separate access layer, either as club members or as casual users [1].

This hierarchy is **the author's requirements model**, taken from the approved proposal. It has not been confirmed as the institute's official organisation chart. The design therefore does not depend on it for authorisation: the `reportsTo` relation in `packages/shared/src/roles.ts` is used only to draw the organisation page, and permissions are granted per role explicitly (Section 4.9).

3.4 User Roles and Responsibilities

Table 3.1 lists the implemented roles. The permission counts are the default grants measured on 12 November 2026 from `/api/auth/me` for each seeded role account. Administrators can change grants at run time.

Table 3.1: *Implemented roles and default permission counts (measured)*

Role (enum)	Responsibility in SportsOS	Scope	Default perms
DIRECTOR	Institute-level oversight; read-only dashboards	Global	18
DEAN_SW	Analytics, escalated complaints, privileged-role approval	Global	27
SPEC_PRESIDENT	Approves event proposals and announcements	Global	22
SPORTS_OFFICER	Facilities, bookings, inventory, staff tasks, analytics	Global	61
GENERAL_SECRETARY	Student lead; submits proposals and requests	Global	16
EVENT_COMMITTEE	Tournament planning, fixtures, results	Global	18
COACH	Attendance sessions and rosters for coached clubs only	Coached clubs	17
GROUND_STAFF	Facility status and schedule; assigned tasks	Optional facility	16
EQUIPMENT_MANAGER	Issue/return, stock ledger, equipment requests	Global	17
SECURITY_STAFF	Gate-pass verification, minimal entry log	Global	11
OPERATIONS_STAFF	Operational tasks	Global	15
SPORTS_COUNCIL	Council functions, participation data	Global	16
SPORT_COORDINATOR	Sessions and rosters for one sport	Sport (required)	17
HOSTEL_SECRETARY	Registers teams for own hostel	Hostel (required)	14
MEDIA_NOMINEE	Drafts announcements (published after approval)	Global	20
CLUB_MEMBER	Club bookings, verified check-in	Club (required)	13
STUDENT	Casual bookings, complaints, own records	Self	12
ADMIN	Users, roles, permission matrix, imports (no operational approvals)	Global	22

3.5 Functional Requirements

Table 3.2: *Functional requirements and current status*

ID	Requirement	Obj.	Status
FR-01	Users sign in with email and password; institute-domain self-registration yields the STUDENT role only.	O3	Implemented, tested
FR-02	Users can recover access through a self-service password reset.	O5	Not implemented
FR-03	Administrators assign scoped, optionally time-bound roles; privileged roles require approval by a second person.	O3	Implemented, tested
FR-04	The role→permission matrix is editable at run time without losing administrator access.	O3	Implemented, tested
FR-05	Users browse facilities and the availability grid; authorised staff change facility status.	O1	Implemented, tested

ID	Requirement	Obj.	Status
FR-06	Students request bookings, which are validated for hours, capacity, horizon and overlap and either auto-approved or queued for approval.	O1, O2	Implemented, tested
FR-07	Each approved booking yields a QR gate pass that security staff verify.	O1	Implemented, tested
FR-08	Coaches open attendance sessions; members check in with a rotating OTP and a GPS fix inside the facility geofence.	O1	Implemented, tested
FR-09	Coaches correct attendance manually with a reason, and the correction history is kept.	O2	Implemented, tested
FR-10	Equipment is issued and returned by QR code or ID, with damage and loss; every stock change is recorded in a ledger.	O1	Implemented, tested
FR-11	Low-stock and overdue-loan alerts are raised automatically.	O2	Implemented, tested
FR-12	Tournament proposals are approved, teams registered (hostel-scoped), fixtures generated and results recorded.	O1	Implemented, tested
FR-13	Complaints follow an assign → resolve → close/reopen lifecycle with escalation and attachments.	O2	Implemented, tested
FR-14	In-app notifications, club announcements and approval-gated official announcements are supported.	O5	Implemented, tested
FR-15	Dashboards show KPIs, participation, utilisation, a demand forecast, equipment wear and attendance integrity.	O4	Implemented, tested (synthetic data)
FR-16	An assistant answers questions and drafts reports from permission-filtered ERP data.	O6	Partial: offline mode tested
FR-17	A mobile app supports check-in, QR scanning, bookings and gate passes.	O5	Partial: type-checked only
FR-18	Every state change is appended to a tamper-evident audit ledger that can be verified.	O2, O6	Implemented, tested

3.6 Non-Functional Requirements

Table 3.3: *Non-functional requirements (ISO/IEC 25010 characteristics)*

ID	Characteristic	Requirement	Evidence / status
NFR-01	Security — confidentiality	Password hashes, OTP secrets and device IDs are never returned by the API.	Verified (smoke test, query omits)
NFR-02	Security — authenticity	Every protected route requires a valid, unexpired JWT; role changes take effect on the next request.	Verified (e2e)
NFR-03	Security — integrity	Stock never goes negative; no double-booking under concurrent requests.	Verified at small scale (6/8 parallel)
NFR-04	Security — accountability	State changes are written to a hash-chained ledger in the same transaction.	Verified (chain valid, 589 entries)
NFR-05	Reliability	Scheduled jobs close stale sessions, complete bookings and flag overdue loans.	Implemented; not separately tested
NFR-06	Maintainability	Typed end to end; shared role/permission definitions; lint-clean.	Verified (tsc 0 errors; ESLint 0 errors, 2 warnings)
NFR-07	Portability	Containerised API, web and database.	Configured; not executed (no Docker daemon)
NFR-08	Performance efficiency	Interactive pages respond within 2 s for 500 concurrent users.	Not tested (load testing planned)
NFR-09	Usability	Role-specific navigation; mobile-first check-in.	Verified per role (UI test); no user study yet
NFR-10	Privacy	Security staff see minimal identity data; the AI layer never receives personal records.	Verified (e2e, AI isolation)

3.7 Use Case Analysis

Figure 3.1 summarises the principal use cases by actor. Only actors and use cases that exist in the implementation are shown. Several actors (for example Ground Staff, Operations Staff, Sports Council, Sport Coordinator, Hostel Secretary and Media Nominee) are omitted from the diagram for legibility; their use cases appear in Table 3.1.

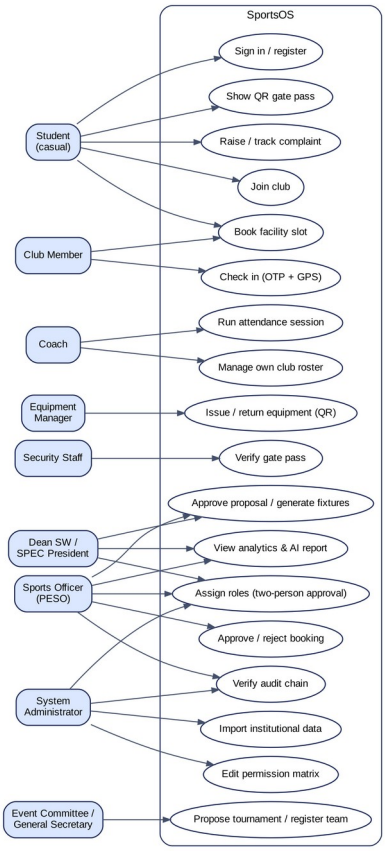


Figure 3.1: *Use case diagram of principal SportsOS actors and use cases (derived from implemented routes)*

3.8 User Stories and Workflows

Requirements were also captured as user stories in the form "As a ⟨role⟩, I want ⟨goal⟩ so that ⟨benefit⟩" [24]. Representative stories, each with its acceptance criterion as implemented, are listed below.

Table 3.4: *Representative user stories with acceptance criteria*

Story	Acceptance criterion (as tested)
As a student, I want to book a free court slot so that I can play without asking in a chat group.	A slot within opening hours on an open facility is approved immediately; an overlapping request receives HTTP 409.
As a coach, I want attendance that cannot be marked by proxy so that practice hours are trustworthy.	Check-in requires the current TOTP, GPS accuracy ≤ 100 m and a position inside the geofence; a device already used by another student is rejected.
As an equipment manager, I want stock that reconciles so that losses are visible.	Ledger deltas sum to the current stock; over-issue receives HTTP 409.
As a hostel sports secretary, I want to register my hostel's team only.	Registering for another hostel is refused; a non-resident player is rejected.
As the Dean (SW), I want escalated complaints brought to me.	An officer's escalation makes the complaint visible in the Dean's queue.

The workflows behind these stories, namely booking, attendance and equipment issue and return, are modelled as activity diagrams in Chapter 5 (Figures 5.2, 5.3 and 5.5).

3.9 System Constraints

- **Institutional data:** no real personnel or student data may be used without authorisation. All seeded data is synthetic, and real data can enter only through an administrator-controlled import with provenance fields.
- **Network:** campus users share a small number of public IP addresses behind NAT, so rate limiting is keyed per account rather than per IP.
- **Location:** GPS accuracy indoors and near large structures can be poor. The geofence therefore tolerates up to 25 m of reported inaccuracy and rejects fixes worse than 100 m.
- **Hosting:** no institute server has yet been allocated; development and testing use local containers.
- **External services:** AI providers require API keys, which have not been provisioned. The system must work without them.

3.10 Development Methodology

The project follows an iterative, incremental process [25] organised around the monthly milestones of the proposal. It is framed methodologically as **design science research** [26]: the artefact (SportsOS) is built to address an identified organisational problem and is evaluated rigorously, and the knowledge produced (requirements model, architecture, verification approach) is made explicit. Each increment followed the same loop:

1. Refine the requirements for a module.
2. Extend the Prisma schema and generate a migration.
3. Implement the service and controller with permission decorators.
4. Build the web page and, where relevant, the mobile screen.
5. Add unit, smoke or e2e tests.
6. Re-run the full verification suite.

Version control uses Git with feature commits merged through a pull request.

CHAPTER 4

SYSTEM ARCHITECTURE AND DATABASE DESIGN

4.1 System Overview

SportsOS is a three-tier system (Figure 4.1). Two clients, a Next.js web application and an Expo/React Native mobile application, call a NestJS REST API. The API persists all state in PostgreSQL through Prisma. A shared TypeScript package defines roles, permissions and wire types once for the API and the web client. Optional external services (the Claude API and Pinecone) are called only when keys are configured. In the evaluated configuration neither was configured, and the system used its built-in offline path.

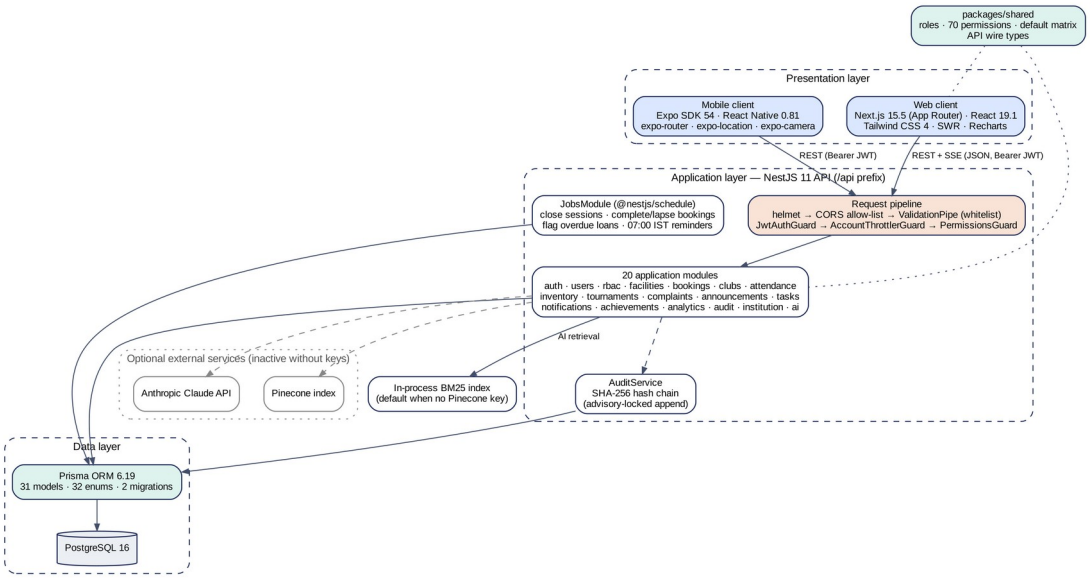


Figure 4.1: *System architecture of SportsOS (all versions verified from package manifests)*

4.2 Architectural Approach

The system follows a **layered, modular-monolith** architecture [27]. The API is one deployable process divided into 20 application modules, each owning a controller (HTTP surface), a service (business rules and transactions) and DTOs (validation). Cross-cutting concerns are implemented once: authentication, rate limiting and authorisation as global guards; validation as a global pipe; and auditing as a service that other modules call inside their own transactions.

A modular monolith was chosen over microservices for three reasons:

1. The modules share one transactional database. Bookings, attendance and the audit ledger must commit atomically together, which distributed services would make considerably harder.
2. The expected load of one institute does not justify distributed deployment.
3. A single deployable keeps operations within the reach of a small campus IT team.

The design keeps module boundaries clean, so a module could later be extracted if needed.

4.3 Frontend Architecture

The web client uses the Next.js 15 **App Router** [28]. All authenticated pages sit in an `(app)` route group whose layout renders the application shell: role-aware navigation, a notification bell and an active-role switcher. The production build verified in this evaluation produced 28 routes (24 static and 4 dynamic: `/clubs/[id]`, `/events/[id]`, `/facilities/[id]`, `/users/[id]`).

Data fetching uses SWR hooks over a thin `fetch` wrapper (`lib/api.ts`). The wrapper attaches the bearer token and the `X-Active-Role` header and normalises errors. Polling intervals reflect operational needs: 4 s for the attendance projector, 15 s for session lists, and 30 s for approval queues and the unread-notification count. Charts use Recharts, and styling uses Tailwind CSS 4.

Navigation items are declared with the permissions they require (`components/shell/nav.ts`), so the menu follows the live permission matrix rather than hard-coded role names. Hiding a menu item is a usability measure only; every action is enforced again on the server.

4.4 Backend Architecture

The API is a NestJS 11 application with the global prefix `/api` [29]. In `main.ts` it installs:

- `helmet` for security headers;
- a CORS allow-list read from configuration;
- a global `ValidationPipe` with `whitelist` and `forbidNonWhitelisted`, so unknown request fields cause HTTP 400 and mass assignment is prevented.

`AppModule` registers three global guards in a deliberate order: `JwtAuthGuard` (authenticate), then `AccountThrottlerGuard` (rate-limit per authenticated account), then `PermissionsGuard` (authorise). The other framework modules are `ConfigModule`,

`JwtModule` (which refuses to start if `JWT_SECRET` is shorter than 16 characters), `ThrottlerModule` and `ScheduleModule`.

Business rules live in services, which use Prisma interactive transactions (`$transaction`) whenever more than one row must change atomically.

4.5 API Design

The API is resource-oriented REST with JSON bodies. Domain actions that are not plain updates are modelled as sub-resource POSTs, for example `POST /bookings/:id/approve`, `POST /inventory/issues/:id/return` and `POST /complaints/:id/escalate`. These actions map one-to-one onto state transitions and audit entries.

A static count of route decorators across the controllers gives **143 route handlers**. Server-Sent Events (SSE) are used for one endpoint, `POST /ai/ask/stream`, which streams the assistant's answer. Appendix B summarises the API surface per module (Table B.1) and gives selected endpoint details (Table B.2).

4.6 Database Architecture

The schema (`apps/api/prisma/schema.prisma`) defines **31 models and 32 enumerations**. Two migrations build it: `20261022055751_init` and `20261104160242_rbac_workflows_institutional_data`. The models fall into eight groups:

- identity and access control (`User`, `RoleAssignment`, `RolePermission`, `PermissionDefinition`);
- facilities and bookings;
- clubs and memberships;
- verified attendance (`AttendanceSession`, `AttendanceRecord`, `AttendanceAttempt`, `AttendanceCorrection`);
- inventory (`Equipment`, `EquipmentIssue`, `InventoryTransaction`, `EquipmentRequest`);
- tournaments (`Tournament`, `Team`, `TeamMember`, `Match`);
- engagement (notifications, complaints with attachments, announcements, tasks, achievements);

- governance (`AuditLog`, `AiUsageLog`, and institutional records with import batches).

Integrity is enforced by the database wherever possible rather than by application code alone, through four mechanisms.

Uniqueness constraints encode business rules. `@@unique([sessionId, userId])` on attendance records means a student is recorded at most once per session even when two requests race. `@@unique([clubId, userId])` prevents duplicate memberships, and `@@unique([tournamentId, round, position])` fixes each match's place in a bracket.

Advisory locks serialise critical sections [14]. Booking creation takes `pg_advisory_xact_lock(hashtext('booking:'||facilityId))` before its overlap check, so two concurrent requests for one facility cannot both pass the check. The audit append takes a constant lock, so the hash chain cannot fork.

Conditional updates keep stock non-negative. Every stock change goes through `applyStock`, which issues `UPDATE … WHERE availableQty >= n` (and the equivalent for damaged and total quantities) and writes the matching ledger row with the resulting balances in the same transaction.

History instead of deletion preserves accountability. Role assignments are never deleted: revocation and expiry change their status, so the table is also the role history. Attendance corrections, inventory transactions and import-batch items are likewise append-only.

4.7 Entity Relationship Diagram

Figure 4.2 shows the core entities and relationships, drawn from the Prisma schema. Only key attributes are shown; the full model list is in Appendix C.

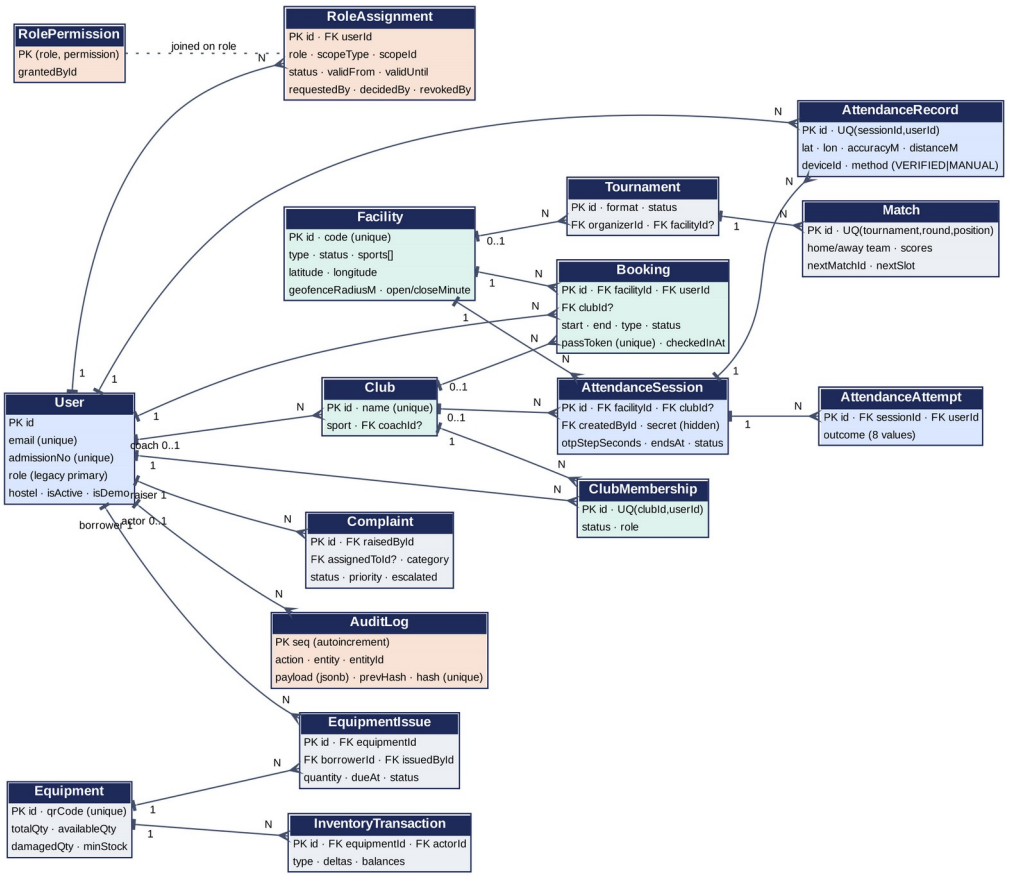


Figure 4.2: *Entity relationship diagram of core SportsOS entities (crow's-foot notation, from schema.prisma)*

4.8 Authentication Architecture

Authentication is password-based with stateless JWT access tokens (Figure 4.3). `POST /auth/login` is public and throttled to 10 attempts per minute per (IP, email) pair. The service always runs a bcrypt comparison, using a fixed dummy hash when the email is unknown, so response time does not reveal which accounts exist. Inactive accounts are refused.

The issued token carries only the user ID and a nominal role and expires after 12 hours. On **every** subsequent request, `JwtAuthGuard` verifies the signature and expiry and then rebuilds the principal from the database. The role claim inside the token is ignored, so deactivation, revocation and permission changes take effect on the next request without waiting for the token to expire.

Self-registration is restricted to the configured institute email domain and always produces the `STUDENT` role. The web client stores the token in `localStorage`; the mobile client stores it in the platform keystore through `expo-secure-store`.

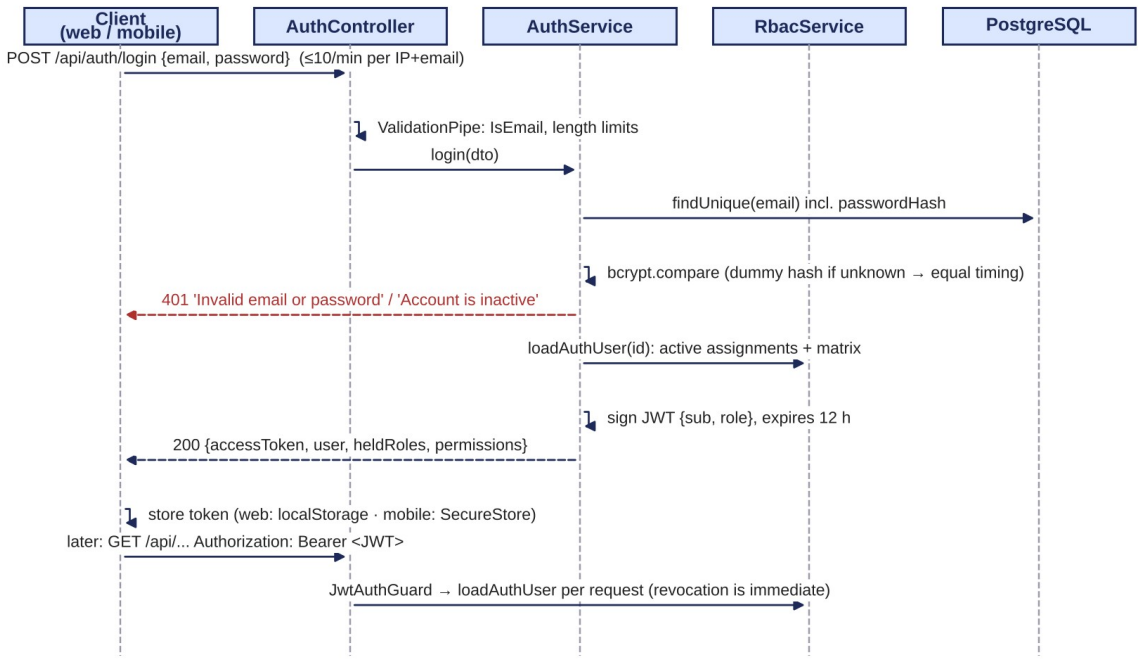


Figure 4.3: *Authentication sequence (login and subsequent authenticated request)*

4.9 Role-Based Access Control Design

The RBAC design (Figure 4.4) separates three concepts: **roles** (18 enumerated values), **permissions** (70 dotted keys such as `booking.approve`), and **assignments**. An assignment is a row linking a user to a role with a scope type (`GLOBAL`, `CLUB`, `HOSTEL`, `SPORT` or `FACILITY`), a scope identifier, a validity window and a status (`PENDING_APPROVAL`, `ACTIVE`, `REJECTED`, `REVOKED` or `EXPIRED`).

The role → permission matrix is stored in `RolePermission` and cached in memory for at most 30 seconds. On first start the API registers 70 permission definitions with **346 default grants** from `packages/shared`. Later edits by an administrator persist across restarts.

Permission resolution is a pure function (`rbac/access.ts`), which makes it unit-testable. It proceeds in three steps:

1. Keep only effective assignments, meaning those that are ACTIVE and inside their validity window.
2. If the client sent an `X-Active-Role` header, the role must be one the user holds, otherwise the request receives HTTP 403. Only that role's grants then apply.
3. The user's permissions are the union of the matrix rows of the remaining grants.

Route-level checks (`@RequirePermission` / `@RequireAnyPermission`) are enforced by `PermissionsGuard`. Object-level checks use `assertCanIn(user, permission,`

`target`), which also tests whether the target lies within the permission's scope. For coaches, club-scoped permissions (`attendance.manage`, `attendance.view_team`, `club.manage_own`, `club.announce`) apply only to clubs in which the user is recorded as coach. Roles that *require* a scope (`CLUB_MEMBER`, `HOSTEL_SECRETARY`, `SPORT_COORDINATOR`) never grant globally, even if mis-assigned without one.

Privileged roles (`ADMIN`, `DIRECTOR`, `DEAN_SW`, `SPEC_PRESIDENT`, `SPORTS_OFFICER`) are created as `PENDING_APPROVAL` and become effective only after approval by a different person who holds `role.approve`. Administrator lock-out is prevented: the permissions that let an administrator manage permissions cannot be removed from the `ADMIN` role.

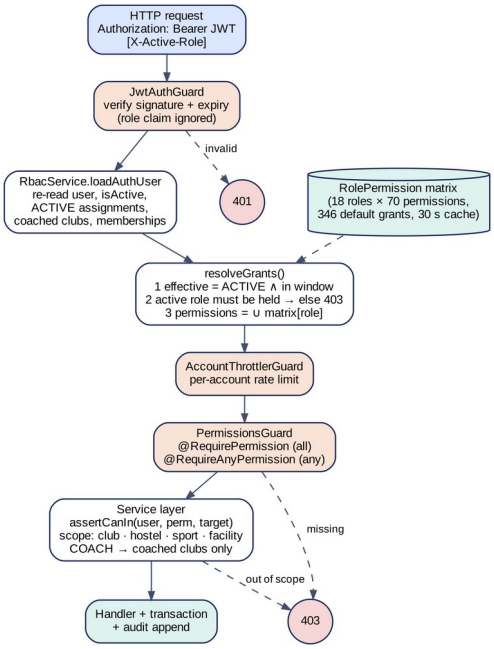


Figure 4.4: *RBAC enforcement pipeline from request to handler*

4.10 Data Flow Diagrams

Figure 4.5 is a level-1 data flow diagram of the five main processes, their external entities and the data stores they read and write. The booking, attendance and inventory processes all append to the audit ledger (D5), and the analytics process reads from the operational stores only.

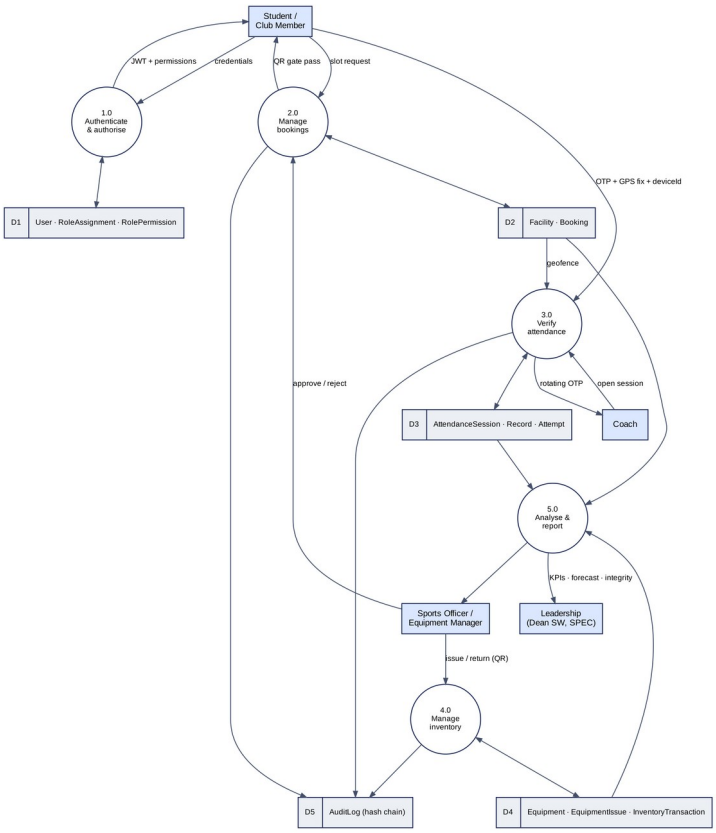


Figure 4.5: *Level-1 data flow diagram*

4.11 Security and Privacy Considerations

Table 4.1 maps the implemented controls to the relevant ASVS 5.0.0 verification areas [9]. The mapping is indicative: a formal ASVS assessment has **not** been carried out, and the table makes no claim of ASVS compliance.

Table 4.1: *Security controls mapped to ASVS 5.0.0 areas (indicative; not an ASVS assessment)*

Area	Implemented control	Verification	Gap
Authentication	bcrypt (12 rounds); timing-equalised login; per-(IP,email) login throttle; inactive accounts refused	Tested	No password reset; no MFA for staff
Session management	12 h JWT; principal reloaded on each request; deactivated user's token rejected	Tested	Web token in localStorage (XSS exposure); no refresh/revocation list
Authorisation	Global deny-by-decorator guards; object-level scope checks; active-role spoofing refused	Tested (18 roles)	No independent penetration test
Validation & encoding	Whitelisting DTO validation; Prisma parameterised queries; upload content sniffing	Tested	—
Configuration	helmet headers (CSP, HSTS, X-Frame-Options, nosniff, Referrer-Policy); CORS allow-list	Observed via curl	Dependency advisories open (Sec. 7.5)
Data protection	Secrets and hashes omitted from queries; minimal entry-log data; AI sees aggregates only	Tested	No encryption at rest configured
Logging & audit	Hash-chained AuditLog with `/audit/verify`; AI usage log stores metadata only	Tested	No external log shipping

On privacy, the seed refuses to wipe a database containing non-demo users. Every demo record carries an `isDemo` flag and visible `[DEMO]` labels. Institutional records are kept separate from user accounts and carry a provenance status (`VERIFIED`, `PENDING_VERIFICATION`, `UNVERIFIED`, `EXPIRED`), and students see only `VERIFIED` records.

4.12 Infrastructure and Deployment Architecture

Deployment is defined in `docker-compose.yml` and two multi-stage Dockerfiles (Figure 4.6). The stack comprises PostgreSQL 16.4 (Alpine) with a health check and a named volume; the API on `node:22-alpine`, which applies pending migrations (`prisma migrate deploy`) before starting; and the Next.js production server.

The proposal's cloud target (AWS with Vercel) has **not** been provisioned. In this evaluation, the Docker daemon was unavailable in the test environment, so the containers were not built. The same components were instead run directly: PostgreSQL 16, the compiled API (`node dist/main.js`) and the Next.js production server (`next start`). The container configuration is therefore reported as *configured but not verified*.

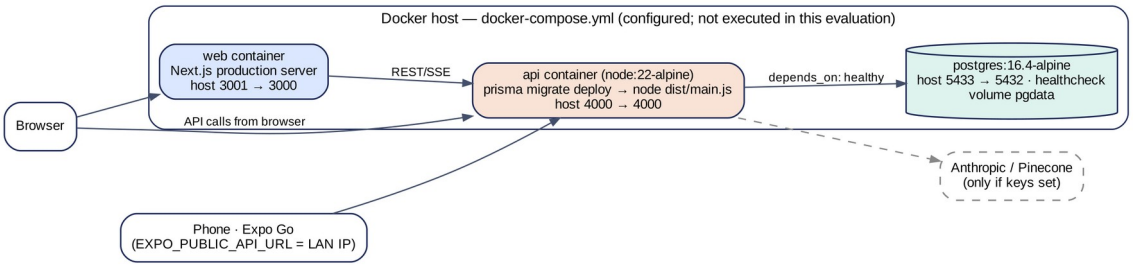


Figure 4.6: *Deployment architecture as configured in docker-compose.yml*

CHAPTER 5

IMPLEMENTATION AND DEVELOPMENT

5.1 Development Environment

Development and verification used Node.js 22.23 (the project requires Node ≥ 20), npm 10.9 workspaces, TypeScript 5.9.3 and PostgreSQL 16. The tooling also included Jest 29.7 for unit tests, Playwright 1.63 with Chromium for browser tests, ESLint 9 with typescript-eslint, and Git. The verification run on 12 November 2026 used a Linux container, a fresh database created with `prisma migrate deploy`, and the project's deterministic synthetic seed.

5.2 Technology Stack and Justification

Table 5.1: *Technology stack: versions verified from package manifests, with the role each plays in SportsOS*

Layer	Technology (version)	How SportsOS uses it	Proposal
Web client	Next.js 15.5.28, React 19.1.10	App Router route group for authenticated pages; static prerender of 24 routes	Next.js 15 ✓
Styling / data	Tailwind CSS 4.3.3, SWR 2.5.1, Recharts 3.10.1	Utility styling; polling hooks; analytics charts	—
Mobile	Expo SDK ~54.0.38, React Native 0.81.5, expo-router 6	Check-in (expo-location), QR scan (expo-camera), keystore (expo-secure-store)	React Native + Expo ✓
API	NestJS 11.2.7 (@nestjs/jwt 11.0.2, throttler 6.7.1, schedule 6.1.3)	Modules, global guards, cron jobs	NestJS ✓
Validation / security	class-validator 0.14.4, helmet 8.3.0, bcryptjs 3.0.3	DTO whitelisting, headers, password hashing	—
ORM / DB	Prisma 6.19.3; PostgreSQL 16	Typed client, migrations, interactive transactions; advisory locks via raw SQL	Prisma + PostgreSQL ✓
Shared code	@sportsos/shared (TypeScript)	Single source for roles, permissions, default matrix, wire types	—
AI (optional)	@anthropic-ai/sdk 1.1.0; @pinecone-database/pinecone 9.1.0	LLM answers and reports; vector index (both inactive without keys)	OpenAI/Claude + Pinecone (partial)
Other	qrcode 1.5.4, exceljs 4.4.0	Gate-pass and item QR images; analytics export	—
Infrastructure	Docker Compose; node:22-alpine; postgres:16.4-alpine	Local container stack	AWS + Vercel (not started)

TypeScript across all tiers allows role and permission identifiers to be checked at compile time on both sides of the API. Prisma was preferred to a raw query builder for its generated types and migration history. Raw SQL is still used where PostgreSQL-specific primitives

are needed, namely the advisory locks [30]. Redis, which an earlier review of the project considered, is **not** part of the implementation: rate-limit counters and the permission-matrix cache are held in process (Section 7.5).

5.3 Monorepo Structure

The repository is an npm-workspaces monorepo. `apps/api`, `apps/web` and `packages/shared` are workspaces. `apps/mobile` is installed separately so that Expo can pin its own React version (19.1.0, against 19.1.10 on the web). Measured source sizes, excluding dependencies and generated files, are given in Appendix A (Table A.1), together with the directory tree.

5.4 Frontend Implementation

The web client implements 24 page files under `app/(app)` together with the public login and registration pages. A role-aware dashboard (`components/role-dashboard.tsx`) assembles different work queues per role:

- pending approvals and live attendance sessions for the Sports Officer (Figure 5.1);
- upcoming bookings for students;
- the day's entry log for security staff.

Shared UI primitives (`components/ui.tsx`) provide consistent cards, tables, forms and empty states. Every page that shows synthetic data carries a visible "DEMO DATA" badge.

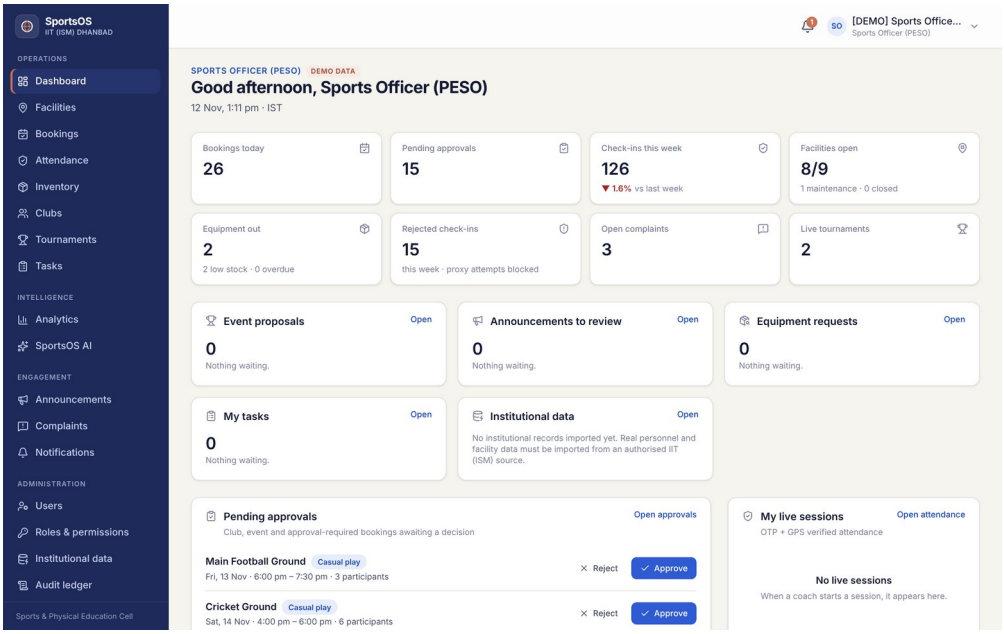


Figure 5.1: *Sports Officer dashboard with KPIs and the approval queue (synthetic demo data)*

5.5 Backend Implementation

Each feature module follows the pattern controller → service → Prisma. Controllers declare permissions declaratively; for example, `@RequirePermission('booking.approve')` on the approve route. Services enforce object-level rules (ownership, scope and state preconditions) and open a transaction for any multi-row change. Every state-changing operation calls `AuditService.append(tx, …)` within that transaction, so the ledger entry and the change commit or roll back together.

Domain errors are mapped to precise HTTP statuses, and the tests in Chapter 6 assert these exact codes:

- `404` when a resource is hidden from the caller;
- `403` when the caller lacks a permission or scope;
- `409` for state conflicts such as an overlapping slot, insufficient stock or a double approval;
- `400` for invalid input.

Four scheduled jobs (`jobs.service.ts`) keep records truthful without manual effort:

- every 5 minutes, expired attendance sessions are closed;
- every 10 minutes, past approved bookings are completed and unreviewed requests whose slot has started are lapsed, with a notification to the requester;
- every 15 minutes, overdue loans are flagged;
- at 07:00 IST daily, booking reminders are sent.

5.6 Database Implementation

The deterministic seed (`prisma/seed.ts`) generates a realistic **synthetic** data set. In the verified run it created 79 users (19 named role accounts plus synthetic students), 9 facilities, 5 clubs, 2,798 bookings, 120 attendance sessions with 1,080 check-ins, 15 equipment items with 283 loans, 3 tournaments and 588 audit entries.

All seeded records are flagged `isDemo`. Users have `@sportsos.demo` emails and `ZZ` admission numbers, and hostels are named "Hostel A–I". Facility coordinates are placeholders near the campus and must be surveyed before geofences are relied on. The seed refuses to reset a database that contains non-demo users unless explicitly forced. The numbers that appear in dashboards and analytics in this report come from this synthetic data and **do not describe real usage at IIT (ISM)**.

5.7 Authentication and User Management

Login, registration and session retrieval (`/auth/me`) are implemented as described in Section 4.8. User administration (`/users`) lets holders of `user.create` or `user.update` create accounts, edit profiles and deactivate users. Roles cannot be set through the user-update endpoint: role changes go only through RBAC assignments, a rule asserted by the smoke test. During evaluation, all 19 seeded role accounts signed in successfully (Appendix F, Table F.2).

Limitation: there is no forgot-password or reset-password flow. `POST /auth/forgot-password` returns 404, and no UI exists. Implementing it requires an email channel, which in turn requires institute mail integration or an approved provider.

5.8 Role-Based Access Control

The design of Section 4.9 is implemented in `apps/api/src/rbac` and three supporting files:

- `access.ts`: pure resolution and scope logic;
- `rbac.service.ts`: principal loading and matrix administration;
- `role-assignments.service.ts`: assignment, approval and revocation, with per-user advisory locks.

The web client provides an administration page with a live role × permission matrix of 1,260 cells (18 × 70), shown in Appendix E (Figure E.1). It also provides a roles drawer for assigning scoped and temporary roles and viewing their history, and an active-role switcher for users who hold several roles. The organisation page draws the proposal's hierarchy and shows the live matrix.

5.9 Sports Club Management

Clubs have a sport, an optional coach and members. Each membership has a status (`PENDING`, `ACTIVE`, `REJECTED`, `LEFT`) and a role (`MEMBER`, `CAPTAIN`, `VICE_CAPTAIN`). Students request to join, and the club's coach or a manager approves the request. Coaches see and edit only the rosters of clubs they coach, post club announcements (delivered as notifications to active members) and read a club calendar.

In the seed, only the Football and Basketball clubs have an assigned coach; the Athletics, Badminton and Cricket clubs have none. The coach-scope rule therefore currently applies

to two clubs, and the cricket club is managed through the sport-scoped coordinator role instead. This was confirmed against the running API.

5.10 Facility and Ground Booking

Each facility has a type, a status, the sports it supports, opening hours (stored as minutes from midnight IST), a slot length, a capacity, an approval flag, and GPS centre and radius. `/facilities/:id/availability` returns the slot grid for a day. The web facility page renders this grid (Appendix E, Figure E.2), and `ground staff` can change a facility's status within their assigned facilities.

Booking creation implements the workflow of Figure 5.2. It validates the slot (duration, horizon, hours, capacity and facility status) and the booking type, then locks the facility and checks for overlaps. Casual bookings on facilities without the approval flag are approved immediately. Others enter the approval queue, and approvers are notified.

An approved booking carries a unique `passToken` rendered as a QR gate pass. Security staff scan it at the gate, and entry is accepted from 15 minutes before the slot. Owners can cancel or reschedule their own bookings; cancelling someone else's booking requires `booking.cancel` and a reason.

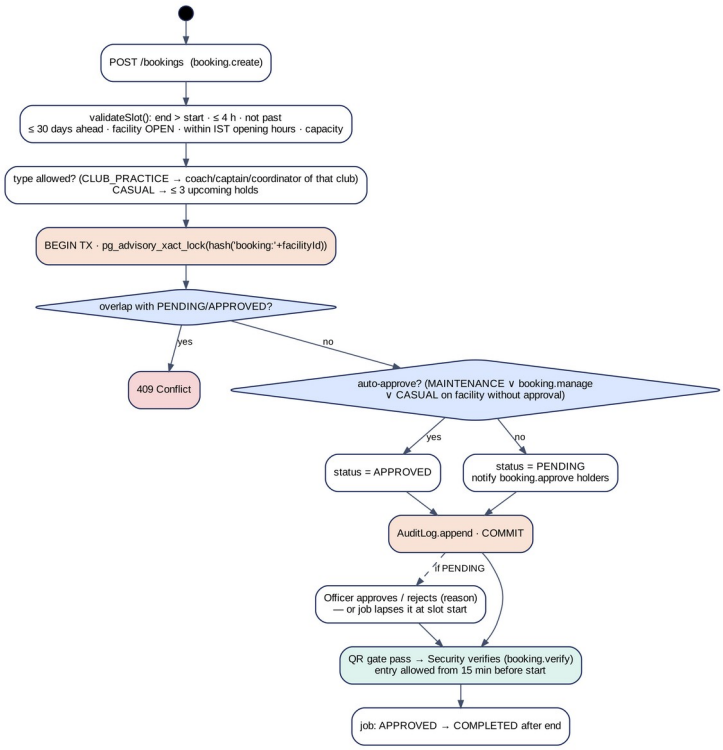


Figure 5.2: *Facility booking workflow (activity diagram from bookings.service.ts)*

5.11 Attendance Management

Verified attendance is the project's most distinctive mechanism. A coach or sport coordinator opens a session for a facility, optionally restricted to one club. The session stores a random 160-bit secret that the API never returns. The coach's screen polls `GET /attendance/sessions/:id/otp` and displays a 6-digit code that changes every 30 seconds. The code is computed as HOTP over a time-step counter (TOTP) [20], [21], and the implementation is unit-tested against the RFC test vectors.

A student's check-in submits the code, a GPS fix with its reported accuracy, and a device identifier. The server evaluates the checks in the order shown in Figure 5.3. The geofence test uses the haversine great-circle distance [31] and accepts a fix when the distance is at most the radius plus min(accuracy, 25 m). Every attempt, accepted or rejected, is stored with its outcome; these records feed the integrity analytics. Re-using a device for a second student raises an alert to the coach.

Coaches can add or remove a record manually with a reason. Manual records are labelled `MANUAL` and store no coordinates, so no GPS data is ever fabricated. On mobile, the check-in tab obtains a high-accuracy fix through `expo-location`. Figure 5.4 shows the coach's session console with a live session.

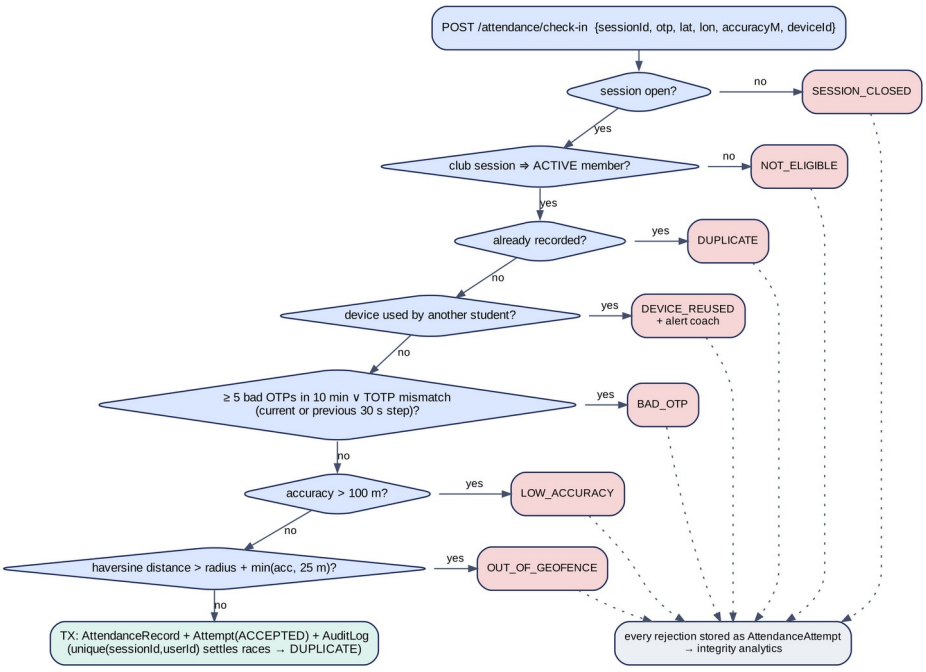


Figure 5.3: *Attendance check-in validation workflow (order as implemented in attendance.service.ts)*

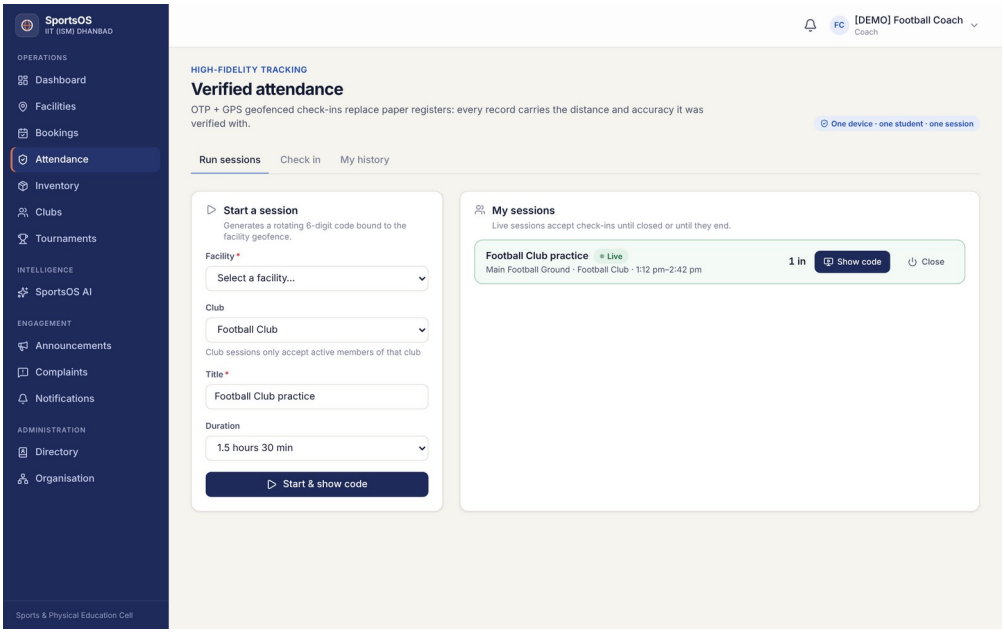


Figure 5.4: *Coach console with a live verified-attendance session (created during evaluation)*

5.12 Equipment and Inventory

Each equipment item carries a unique QR code and the quantities `totalQty`, `availableQty` and `damagedQty`, together with a `minStock` threshold. All quantity changes go through `applyStock` (Section 4.6), which writes an `InventoryTransaction` with deltas and resulting balances. The transaction types are `INITIAL`, `RESTOCK`, `ISSUE`, `RETURN`, `DAMAGE_REPORT`, `REPAIR`, `LOSS`, `WRITE_OFF` and `ADJUSTMENT`.

Issue and return follow Figure 5.5. Items are looked up by QR code or ID, and borrowers by email or admission number. A return records the good, damaged and lost quantities in one transaction. Club members can raise equipment requests, which the equipment manager approves (the requester cannot approve their own request) and fulfils through the same ledger. The inventory screen (Appendix E, Figure E.3) shows stock with low-stock highlighting, and an audit page shows the hash-chained ledger.

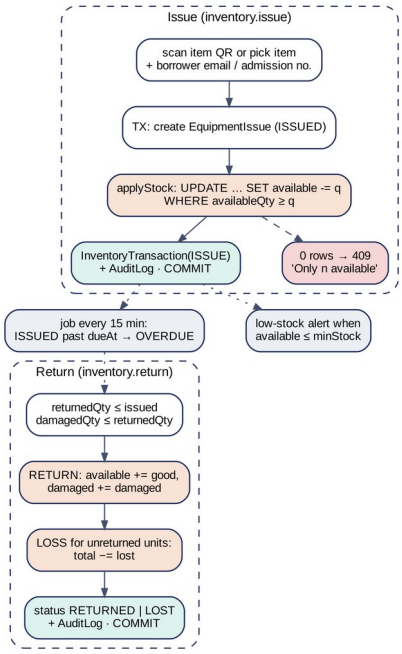


Figure 5.5: *Equipment issue and return workflow*

5.13 Tournament and Event Management

Tournaments move through the statuses `PROPOSED` → (`REJECTED` | `DRAFT`) → `ONGOING` → `COMPLETED`. Proposals from the General Secretary or the Event Committee require approval by a holder of `event.approve`, and the proposer cannot approve their own proposal. Hostel secretaries register teams only for their own hostel and only with resident players; a player can belong to only one team per tournament.

Fixture generation (`tournaments/bracket.ts`) supports two formats. **Single elimination** uses standard seeding, with byes allocated to the top seeds and winners advanced automatically through `nextMatchId` and `nextSlot`. **Round robin** uses the circle method, with standings awarding 3, 1 or 0 points and goal-difference tie-breaks. Both are unit-tested. Figure 5.6 shows a generated knock-out bracket in which two byes are allocated to the top seeds.

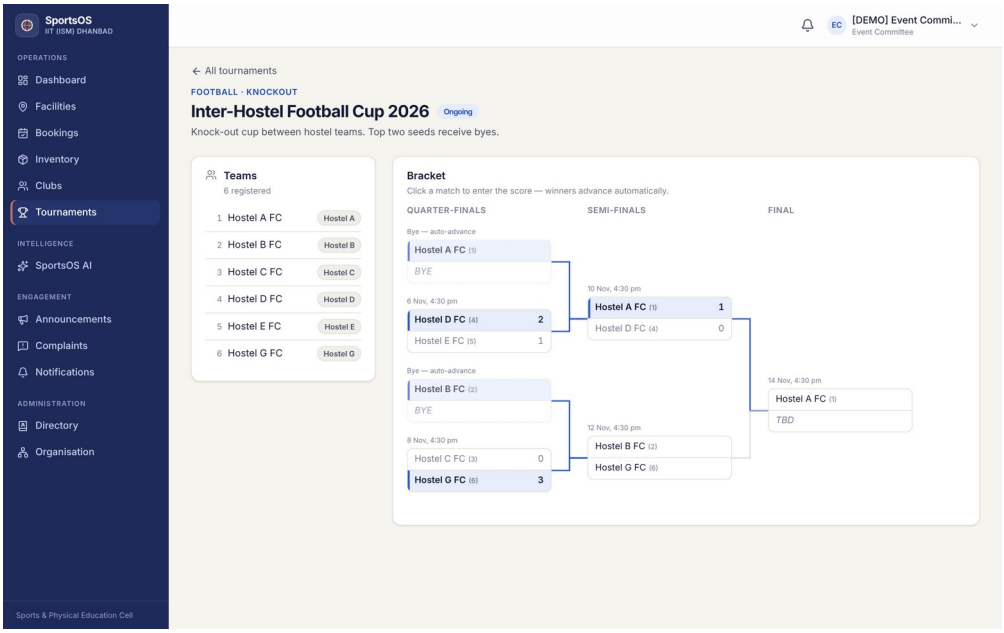


Figure 5.6: *Knock-out bracket with seeded byes and automatic advancement (synthetic tournament)*

5.14 Complaint Management

A complaint has a category, a priority, an optional facility or equipment link and a status that follows the lifecycle in Figure 5.7. The lifecycle enforces five rules:

- resolution requires a note;
- the person who raised the complaint cannot resolve it;
- only the raiser or a complaint manager may close it, and only after resolution;
- reopening is limited in both time and count;
- escalation by a manager makes the complaint visible to the Dean (Students' Welfare).

Image attachments are stored in the database. Their content type is verified by inspecting the file's bytes, so a script renamed as an image is rejected. Only the raiser and the complaint's handlers can download attachments.

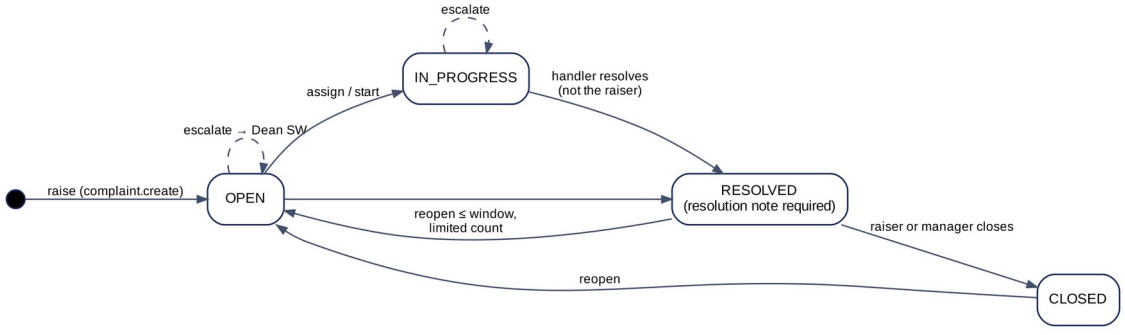


Figure 5.7: *Complaint lifecycle state machine (complaints.service.ts)*

5.15 Notifications and Announcements

Notifications are stored per user with a type (`INFO`, `ALERT`, `APPROVAL`, `REMINDER`) and an optional deep link. Services create notifications on events such as pending approvals, lapsed bookings, device-reuse alerts and low stock. The web client polls the unread count every 30 seconds; there is no push channel (WebSocket or mobile push notifications) yet.

Official announcements follow a draft → submitted → approved → published workflow. A media nominee can draft and publish approved content but cannot approve it, and students see an announcement only after publication. Club announcements are posted directly by the club's coach.

5.16 Dashboard and Analytics

The analytics module exposes several views: KPIs, weekly participation, facility utilisation with a weekday × hour heatmap, equipment wear, attendance integrity (the distribution of rejected-attempt outcomes), complaint statistics, booking trends and an export (Appendix E, Figure E.4). Demand forecasting uses additive Holt–Winters exponential smoothing with a weekly season [32], [33]. The smoothing parameters are chosen by grid search on one-step-ahead squared error, and approximate prediction intervals are reported.

In the verified run, the forecast endpoint executed successfully on the synthetic history. Its in-sample error is **not** reported as a result, because it reflects the seed generator rather than real demand. Forecast accuracy on real booking data is planned evaluation work.

5.17 Mobile Application

The Expo application (`apps/mobile`) uses `expo-router` with five tabs (Home, Bookings, Check-in, Scan, Profile) and further screens for booking a facility, booking details, equipment details by QR code and a full-screen gate pass. It reuses the same API and stores the token with `expo-secure-store`. The device identifier used for device binding comes from `src/device.ts`.

Verification status: the mobile project installed cleanly and passed a strict TypeScript check (`tsc --noEmit`, exit 0) on 12 November 2026. It has **not** been run on a physical device or simulator, so GPS accuracy, camera scanning and usability on phones remain unverified. No mobile screenshots are therefore included.

5.18 AI-Assisted Features and Future Integration

The AI layer (`apps/api/src/ai`) implements retrieval-augmented question answering [34] and executive-report generation. A knowledge service renders the live ERP state into retrieval documents. These documents contain only aggregates such as facility summaries, stock levels and analytics figures, and **never** individual students' personal records. Each document carries the permissions required to read it. Before retrieval, the documents are filtered by the caller's permissions, so a student never receives analytics or integrity documents, and withheld context is never sent to the language model.

Retrieval uses Pinecone when a key is configured. Otherwise it uses an in-process Okapi BM25 index [35]. Answers are generated by the Claude API when a key is configured; otherwise the endpoint returns the retrieved facts and a templated report. Answers stream to the web client over SSE, and the web assistant page includes optional browser speech input through the Web Speech API (`lib/speech.ts`); the assistant page is shown in Appendix E (Figure E.5).

Verification status: in the evaluated configuration, `GET /ai/status` reported `{"llm":"offline","vectorStore":"local","indexedDocuments":30}`. Offline retrieval, SSE streaming, permission isolation and the offline report were tested end to end. The Claude and Pinecone paths are covered only by unit tests with mocked clients, because no keys were available. No claim is made about the quality of LLM-generated answers. Voice input has not been tested.

5.19 Module-Wise Implementation Status

Table 5.2 summarises the status of each module using four labels:

- **Implemented** — the code exists end to end;
- **Tested** — at least one automated test executed and passed on 12 November 2026;
- **Partial** — the module is implemented but a significant part is unverified or missing;
- **Planned** — the module has not been started.

The evidence for each status is listed in Appendix D.

Table 5.2: *Module-wise implementation status at mid-evaluation*

Module	Proposal pillar	Backend	Web	Mobile	Status
Authentication & users	P1	✓	✓	✓ (login)	Implemented, tested
Forgot / reset password	P1	✗	✗	✗	Not implemented

Module	Proposal pillar	Backend	Web	Mobile	Status
RBAC (scoped, time-bound, two-person)	P1	✓	✓	—	Implemented, tested
Facilities & booking, QR gate pass	P1/P2	✓	✓	✓	Implemented, tested
Clubs & rosters	P1	✓	✓	—	Implemented, tested
OTP + GPS attendance	P2	✓	✓	✓	Implemented, tested (API/web)
Equipment & inventory ledger, QR	P2	✓	✓	✓ (scan)	Implemented, tested
Tournaments & fixtures	P3	✓	✓	—	Implemented, tested
Complaints	—	✓	✓	—	Implemented, tested
Notifications & announcements	—	✓	✓	—	Implemented, tested (polling only)
Operational tasks	—	✓	✓	—	Implemented, tested
Dashboards & analytics	P3	✓	✓	—	Implemented, tested (synthetic)
Audit ledger	P2	✓	✓	—	Implemented, tested
Institutional data import	—	✓	✓	—	Implemented, tested
AI assistant & reports	P3	✓	✓	—	Partial: offline tested, LLM mocked
Mobile application	All	—	—	✓	Partial: type-checked, not run
Redis caching / distributed rate limit	—	✗	—	—	Not implemented
Cloud deployment (AWS/Vercel)	Infra	—	—	—	Planned

CHAPTER 6

SOFTWARE TESTING AND EVALUATION

6.1 Introduction

This chapter reports the verification performed for the mid-evaluation. **All results were obtained by the author on 12 November 2026** in a Linux container (Node.js 22.23.4, PostgreSQL 16, Chromium through Playwright 1.63), against a database freshly migrated with `prisma migrate deploy` and seeded with synthetic data. The production builds of the API and web client were used.

The evaluation is **preliminary**. It establishes functional and security correctness at the level of automated tests. It does not establish performance at scale, field behaviour of GPS on phones, or usability with real users; those evaluations are planned (Section 7.8).

6.2 Testing Strategy

Testing follows a layered strategy (Table 6.1). Pure logic with mathematical or security significance is unit-tested in isolation: OTP generation, geofencing, hash chaining, bracket generation, forecasting, import parsing and permission resolution. Whole-system behaviour is tested through the HTTP API against a running server and a real database. User-facing behaviour is tested in a real browser.

Because broken access control is the dominant risk [8], the strategy puts most of its weight on authorisation. Every protected-endpoint probe is executed for every one of the 18 roles. Allowed roles must not receive 401 or 403, and denied roles must receive exactly 403.

Table 6.1: *Test levels, tools and results of the 12 November 2026 run*

Level	Tool / command	Scope	Result
Static analysis	`npm run typecheck`, `npm run lint`	shared, API, web	PASS: 0 type errors; ESLint 0 errors, 2 warnings
Static analysis (mobile)	`npx tsc --noEmit` in apps/mobile	mobile	PASS: exit 0
Schema	`prisma validate`, `prisma migrate deploy`	database	PASS: valid; 2 migrations applied
Unit	`npm test` (Jest)	8 suites	PASS: 71 / 71
API end to end (smoke)	`npm run test:smoke`	all modules	PASS: 83 / 83
RBAC & workflow e2e	`npm run test:e2e`	18 roles × endpoints, workflows	PASS: 200 / 200

Level	Tool / command	Scope	Result
Targeted checks (author)	scripted HTTP checks	logins, attendance, restart, ports	PASS: see Table 6.3, App. F
Browser: per role	`ui-roles.e2e.cjs` (Playwright)	9 roles	PASS: 9 / 9
Browser: flows	`ui-flows.e2e.cjs` (Playwright)	12 interactive checks	PASS: 12 / 12
Production build	`nest build`, `next build`	API, web	PASS: 28 web routes
Container build	`docker compose up --build`	deployment	Not run: Docker daemon unavailable
Load / penetration / device	—	—	Not run (planned)

6.3 Unit Testing

Table 6.2: *Unit test suites (Jest), 71 tests, all passing*

Suite	Tests	What is verified
rbac/access.spec.ts	32	Union of roles; revoked, pending, expired and future assignments ignored; active-role narrowing; spoofed active role rejected; no seniority inheritance; coach/sport/hostel/facility scopes; expected grants and denials for each of the 18 roles
institution/import-parser.spec.ts	11	RFC 4180 CSV parsing (quotes, CRLF, BOM), required columns, provenance rules, duplicate keys
ai/ai.service.spec.ts	8	Offline answers; mocked Claude streaming, refusal and fallback; permission isolation of retrieved documents
tournaments/bracket.spec.ts	5	Seeding and byes, advancement links, round-robin schedule and standings
common/otp.spec.ts	4	RFC 4226 HOTP test vectors; TOTP current/previous-step acceptance
common/geo.spec.ts	4	Haversine distance; inside, outside and low-accuracy verdicts
common/hash-chain.spec.ts	4	Canonical JSON; chain verification; detection of edited and deleted entries
analytics/forecast.spec.ts	3	Holt–Winters fitting and prediction-interval behaviour

6.4 Integration Testing

Integration was tested through the running API with the real PostgreSQL database. Two concurrency tests are especially relevant because they exercise the database-level design of Section 4.6:

1. **Booking race:** six simultaneous requests for one slot returned `201, 409, 409, 409, 409, 409`, so exactly one booking won.
2. **Stock race:** eight simultaneous single-unit issues against three units in stock returned exactly three `201` responses and five `409` responses. Afterwards the ledger deltas reconciled exactly with the stored stock, and the invariant total = available + issued + damaged held.

These tests confirm the locking and conditional-update design at small scale only. Contention behaviour under realistic load has not been measured.

6.5 Functional Testing

Table 6.3 lists representative cases covering the scenarios named in the evaluation brief. The complete case tables (63 cases across functional, authentication, RBAC, database and attendance testing) are given in Appendix F.

Table 6.3: *Representative test cases (complete tables in Appendix F)*

Test ID	Test scenario	Expected result	Actual result	Status
AUTH-01	Successful login, each of the 19 seeded role accounts	HTTP 200 + token	19 / 19 returned 200	PASS
AUTH-02	Invalid login (unknown account)	HTTP 401	401	PASS
AUTH-04	Unauthorised API access (no token)	HTTP 401	401	PASS
RBAC-01	Role-based route protection: student calls analytics	HTTP 403	403	PASS
RBAC-15	Role-based navigation in the browser (9 roles)	Only permitted menu items	9 / 9 as expected	PASS
ATT-01/02	Coach opens a session for own club / for another club	201 / 403	201 / 403	PASS
BK-01	Student books a free slot	APPROVED	APPROVED	PASS
BK-04	Booking conflict: 6 concurrent requests for one slot	Exactly one succeeds	201 ×1, 409 ×5	PASS
ATT-07/05	Attendance: valid check-in / check-in 1.1 km away	ACCEPTED / OUT_OF_GEOFENCE	ACCEPTED (11 m) / OUT_OF_GEOFENCE (1,112 m)	PASS
INV-01/03	Inventory issue by QR, then return with one damaged unit	Stock and ledger updated	Issued; available 13 → 12, damaged +1	PASS
CMP-02	Complaint status: assignee resolves, raiser reopens	RESOLVED → OPEN	As expected	PASS
DB-03	Database persistence across API restart	Counts unchanged	2,798 bookings; 589 audit entries before and after	PASS
AUTH-09	Forgot-password request	Reset flow	404 (not implemented)	Not impl.

6.6 Authentication Testing

AUTH-01 was a specific concern from earlier reviews, which had reported demo sign-in failures. The failure could not be reproduced: after migration and seeding, every account signed in. The project changelog records that the seed script previously did not load the API's `.env` file, so `npm run db:seed` failed locally and the demo accounts were never created. The current seed loads `.env`, which is consistent with the failures having been a seeding problem rather than an authentication defect.

6.7 Authorisation and RBAC Testing

The e2e suite's endpoint enforcement matrix probed 23 protected endpoints with all 18 roles. In every case, allowed roles were not refused and denied roles received exactly 403. Selected cases from this matrix, and from the author's scope tests, are listed in Appendix F (Table F.3).

6.8 Database Testing

6.9 API Testing

Beyond the functional cases above, the API was checked for protocol-level behaviour:

- `GET /api/health` returned `{"status":"ok","db":true}`.
- The SSE endpoint emitted the event sequence `sources, delta, done`.
- SQL-like search input was treated as data.
- Malformed inputs were rejected with 400 before reaching services, for example a non-numeric OTP (ATT-11) and device identifiers shorter than eight characters. The latter was discovered when the author's own first test script used short IDs and was correctly rejected.

Duplicate API processes on port 4000. Earlier reviews reported more than one API process competing for port 4000. This was not reproduced. Exactly one API process was running, and starting a second instance failed immediately with `EADDRINUSE: address already in use 0.0.0.0:4000` and exit code 1, leaving the first instance unaffected. The application therefore cannot serve two processes on one port. The earlier observation most likely came from a stale development process (for example a leftover `nest start --watch`), and a start-up port check in the development scripts is listed as a minor improvement.

6.10 End-to-End Testing

Browser tests drove the production web build with Playwright and Chromium. The per-role test signed in as nine different roles and visited each role's main pages. For each role it asserted the presence of expected navigation items and the absence of forbidden ones, with zero console errors and zero failed API calls; all 9 roles passed.

The flow test passed all 12 interactive checks:

- the active-role switcher narrows the UI, and the choice persists across reload;

- the permission matrix renders;
- a role assigned from the drawer appears in the user's role history;
- an import preview shows one new row and one error;
- a student sees no management controls.

The attendance scenarios that the author executed directly against the running API are listed in Appendix F (Table F.5).

6.11 Security Testing

Security testing at this stage consists of three parts:

1. The authorisation and authentication cases above.
2. An inspection of the HTTP response headers. The API returned Content-Security-Policy, Strict-Transport-Security, X-Frame-Options (SAMEORIGIN), X-Content-Type-Options (nosniff), Referrer-Policy (no-referrer) and Cross-Origin-Opener/Resource-Policy. A request from an origin outside the CORS allow-list received no `Access-Control-Allow-Origin` header.
3. A dependency audit. `npm audit --omit=dev` reported **10 advisories (7 high, 3 moderate)** in production dependencies, including `body-parser`, `multer`, `deepmerge-ts`, the PostCSS bundled with Next.js, and `uuid`. These have not yet been triaged for exploitability in SportsOS's configuration and are recorded as an open item.

No penetration test, fuzzing or formal ASVS assessment has been performed.

6.12 Preliminary Evaluation

Against its functional and authorisation requirements, the prototype passes every automated check that was executed: 71 unit tests, 83 smoke checks, 200 RBAC and workflow checks, 21 browser checks, and and the author's targeted checks in Appendix F (19 account sign-ins, 11 attendance scenarios, and restart, port, audit-chain, header and CORS checks). Two functional gaps were confirmed (no password reset, no Redis-backed shared state), and one cosmetic defect was found during the evaluation. The defect, **D-01**, is in the attendance form: a 90-minute duration is labelled "1.5 hours 30 min" because `attendance/page.tsx` divides minutes by 60 without flooring. It is recorded for correction.

These results support the claim that the core is **functionally correct and enforces its access-control model** at prototype scale. They do **not** yet support claims about the following:

- performance at scale;
- GPS reliability on phones in the field;
- mobile usability;
- AI answer quality;
- real-world adoption.

Each of these is addressed in the revised roadmap.

CHAPTER 7

RESULTS, LIMITATIONS AND FUTURE WORK

7.1 Current Development Status

Between the proposal (August 2026) and this evaluation, the project has moved from a design to a working, tested prototype. It consists of the following:

- a 31-model database with migrations;
- an API of 143 route handlers in 20 modules;
- a 28-route web application;
- a type-checked mobile application;
- a verification suite of more than 370 automated checks.

Measured against the proposal's roadmap, the September to November milestones are met, and the December (tournaments) milestone has been delivered early. The January (AI) and February (mobile) milestones are partially met in code but not yet verified.

7.2 Achievement Against Research Objectives

Table 7.1: *Achievement against the proposal's objectives*

Obj.	Evidence	Assessment
O1	One schema and API for facilities, bookings, clubs, attendance, inventory, tournaments, complaints, announcements and tasks	Met (prototype)
O2	Approval workflows with notifications, four scheduled jobs, automatic conflict detection, import with rollback	Met (prototype)
O3	18 roles, 70 permissions, scoped/time-bound assignments, two-person approval; 200/200 RBAC e2e checks	Met (prototype)
O4	Nine analytics endpoints and dashboards; Holt–Winters forecast	Partly met: synthetic data only
O5	Single web portal for students; mobile client	Partly met: no user study; mobile not device-tested
O6	Modular monolith, containers, migrations, AI layer with graceful degradation	Partly met: no cloud deployment, load test, live AI or shared cache

7.3 Preliminary System Demonstration

The system can be demonstrated end to end with the synthetic data set. A recommended demonstration sequence, each step exercised during this evaluation, is as follows:

1. A student signs in and books a court; the overlapping request is refused.
2. The Sports Officer approves a pending booking from the dashboard.
3. A coach opens a session, the code rotates on screen, and a member checks in; out-of-fence and wrong-code attempts are rejected and appear in the integrity analytics.
4. The equipment manager issues and returns equipment, and the ledger is shown.
5. The Event Committee views a knock-out bracket.
6. An administrator changes the permission matrix and shows the effect immediately.
7. The audit chain is verified.

Additional screenshots are given in Appendix E.

7.4 Engineering Contributions

- **Scope-aware, time-bounded RBAC without implicit hierarchy**: authority is bounded to a club, hostel, sport or facility, temporary roles lapse at their deadline, and privileged roles need a second approver, all within a pure, unit-tested resolution function.
- **Proof-of-presence attendance** combining TOTP, GPS geofencing with accuracy-aware tolerance, device binding, and a complete attempt log that turns rejected attempts into integrity analytics.
- **Database-enforced consistency** for bookings and inventory (advisory locks, conditional updates, unique indexes), together with a **reconciled stock ledger** and a **hash-chained audit log** written in the same transactions.
- **A role-exhaustive verification method** that probes every protected endpoint class with every role and checks the result at both the API and browser levels.

7.5 Current Limitations

Table 7.2: *Known limitations at mid-evaluation*

ID	Limitation	Impact	Planned action
L-01	No forgot/reset-password flow	Users depend on an administrator to regain access	Email-token reset once a mail channel is approved
L-02	Web JWT kept in localStorage	Token theft possible if an XSS flaw appears	httpOnly, SameSite cookie session
L-03	No Redis: rate limits and matrix cache are in process	Limits not shared across API replicas; horizontal scaling unsafe	Redis-backed throttler storage and cache
L-04	10 dependency advisories (7 high)	Potential DoS vectors in request parsing	Triage, upgrade, re-test
L-05	Mobile app not run on devices	GPS and camera behaviour unverified	Device testing on Android and iOS

ID	Limitation	Impact	Planned action
L-06	AI verified offline only	LLM answer quality and latency unknown	Run with keys; evaluate grounding and refusals
L-07	Synthetic data only; placeholder geofences	Analytics and forecast not meaningful yet	Survey grounds; import verified institutional data
L-08	No load, penetration or usability testing	Scalability and usability unproven	k6/Artillery load tests; ASVS review; SUS study
L-09	Notifications by polling	Delay up to 30 s; no phone push	WebSocket/SSE channel; Expo push notifications
L-10	Container stack not executed in evaluation; no cloud	Deployment unverified	Build and run compose; staging deployment
L-11	Defect D-01 (duration label)	Cosmetic	One-line fix (floor division)
L-12	Stakeholder hierarchy not validated with SPEC	Role model may need changes	Requirements-validation workshop

7.6 Technical Challenges

Five technical challenges shaped the design.

Concurrency without a queue. Preventing double bookings could have been attempted with application-level checks, but such checks are unsafe under concurrent requests. Facility-keyed advisory locks inside the booking transaction solved this without additional infrastructure, and the concurrency tests confirm the behaviour.

Closing authorisation defects found in the baseline. An audit of the first working version, recorded in the project changelog, found that the Sports Officer could grant leadership roles to anyone, that any sport coordinator could manage any club, that coaches could list all users with phone numbers, and that a student could obtain integrity analytics through the AI assistant. The RBAC overhaul (commit `f835cbd`) addressed these through scoped assignments, two-person approval, a pure and unit-tested resolution function, and permission-tagged AI documents. Each fix is now asserted by the e2e suite (Section 6.7).

Honest manual corrections. A manual attendance record must not invent coordinates. The schema therefore makes latitude and longitude nullable and adds an explicit method field.

Time zones. Opening hours, reminders and "today" are defined in IST, while timestamps are stored in UTC. Explicit IST helpers in `common/time.ts` avoid off-by-one-day errors.

Evaluation environment. During this evaluation the sandbox removed permissions on the working directory, which terminated PostgreSQL. The database was relocated and restarted, and data was intact on recovery, because committed transactions survive a crash and restart.

7.7 Future Enhancements

- Password reset, optional TOTP-based two-factor authentication for privileged staff, and cookie-based web sessions.
- Redis for shared rate-limit state, caching and a notification fan-out channel; push notifications to the mobile app.
- Live evaluation of the AI assistant: grounding accuracy against database facts, refusal behaviour, latency and cost, and evaluation of voice input.
- Forecast evaluation on real booking history, with rolling-origin validation against seasonal-naïve baselines [33].
- Integration with institute single sign-on and verified institutional data feeds, subject to approval.
- Accessibility review of the web client.

7.8 Revised Development Roadmap

The original nine-month roadmap [1] is retained. Because core modules were completed early, the remaining months are re-planned towards validation and hardening (Figure 7.1).

1. **November–January:** requirements validation and user-acceptance testing with SPEC; ground survey for geofences.
2. **December–January:** security hardening (L-01 to L-04).
3. **January–February:** AI evaluation with live keys.
4. **February:** device testing of the mobile app.
5. **March–April:** load and penetration testing, staging deployment and thesis writing.

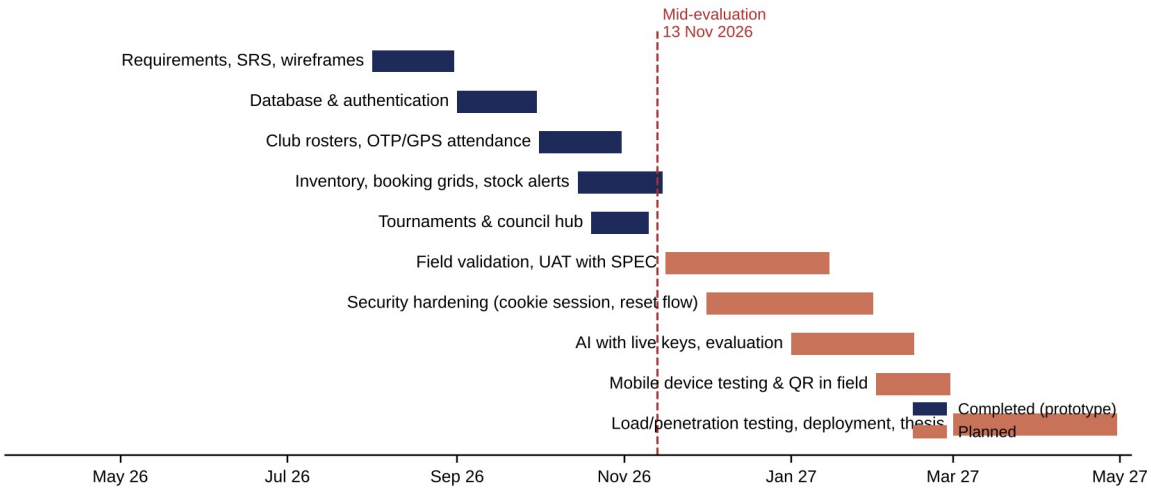


Figure 7.1: *Revised development roadmap (completed work and planned activities)*

7.9 Conclusion

At mid-evaluation, SportsOS is a working prototype rather than a proposal. A single relational model and API now unify facility booking, verified attendance, inventory, tournaments, complaints and analytics for eighteen stakeholder roles.

The central engineering claims have been verified by automated tests executed for this report:

- access is confined to each role's permissions and scopes;
- attendance requires physical presence as defined by OTP, GPS and device checks;
- bookings and stock remain consistent under concurrent requests;
- every state change is captured in a verifiable audit chain.

The limitations are equally clear: no password reset, no shared cache, an untested mobile client, AI verified only offline, synthetic data, and no load, penetration or user testing. The second half of the project is therefore devoted to validating SportsOS with its stakeholders and hardening it for deployment at IIT (ISM) Dhanbad.

REFERENCES

[1] A. Kumar, "SportsOS: An AI-Powered Sports ERP for IIT (ISM) Dhanbad," Integrated M.Tech thesis proposal, Dept. of Mathematics and Computing, IIT (ISM) Dhanbad, Aug. 2026.

[2] Sports & Physical Education Center (SPEC), IIT (ISM) Dhanbad, "Activities." [Online]. Available: https://people.iitism.ac.in/~dsw/spec/activity.html (content consulted through a search-engine index on 12 Nov. 2026; page undated).

[3] Indian Institute of Technology (Indian School of Mines) Dhanbad, "SPEC." [Online]. Available: https://www.iitism.ac.in/spec (host not reachable from the evaluation environment on 12 Nov. 2026).

[4] Indian Institute of Technology (Indian School of Mines) Dhanbad, "Facilities — Dean (Students' Welfare)." [Online]. Available: https://www.iitism.ac.in/facilities-dsw (host not reachable from the evaluation environment on 12 Nov. 2026).

[5] Sports & Physical Education Center (SPEC), IIT (ISM) Dhanbad, "Facilities." [Online]. Available: https://people.iitism.ac.in/~dsw/spec/facilities.html (content consulted through a search-engine index on 12 Nov. 2026; page undated).

[6] L. Haffner, I. Oshri, and J. Kotlarsky, "Directions for future IS research on sports digitalisation: A stakeholder perspective," *The Journal of Strategic Information Systems*, vol. 34, no. 2, Art. no. 101905, 2025, doi: 10.1016/j.jsis.2025.101905.

[7] D. E. W. Saputra and P. Asmawati, "Elevating user experience and service excellence in sports facilities via integrated management information systems: A systematic review," *Sports Science and Health*, vol. 15, no. V, pp. 111–118, 2025, doi: 10.7251/SSH25V111S.

[8] OWASP Foundation, *OWASP Top 10: 2021*. [Online]. Available: https://owasp.org/Top10/

[9] OWASP Foundation, *Application Security Verification Standard (ASVS)*, version 5.0.0, May 2025. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

[10] G. Vial, "Understanding digital transformation: A review and a research agenda," *The Journal of Strategic Information Systems*, vol. 28, no. 2, pp. 118–144, 2019.

[11] M. D. Shahrom and K. A. Mohamad Sukri, "Swimming facilities management system for Universiti Tun Hussein Onn Malaysia Sport Centre with two-factor authentication," *Applied Information Technology and Computer Science*, vol. 6, no. 2, pp. 625–643, 2025.

[12] T. H. Davenport, "Putting the enterprise into the enterprise system," *Harvard Business Review*, vol. 76, no. 4, pp. 121–131, 1998.

[13] H. Klaus, M. Rosemann, and G. G. Gable, "What is ERP?," *Information Systems Frontiers*, vol. 2, no. 2, pp. 141–162, 2000.

[14] The PostgreSQL Global Development Group, "Explicit locking — Advisory locks," *PostgreSQL 16 Documentation*, §13.3.5. [Online]. Available: https://www.postgresql.org/docs/16/explicit-locking.html

[15] R. E. Freeman, *Strategic Management: A Stakeholder Approach*. Boston, MA, USA: Pitman, 1984.

[16] R. Sandhu, D. Ferraiolo, and R. Kuhn, "The NIST model for role-based access control: Towards a unified standard," in *Proc. 5th ACM Workshop on Role-Based Access Control (RBAC '00)*, Berlin, Germany, 2000, pp. 47–63, doi: 10.1145/344287.344301.

[17] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 224–274, 2001.

[18] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference, FREENIX Track*, Monterey, CA, USA, 1999, pp. 81–91.

[19] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[20] D. M'Raihi, M. Bellare, F. Hoornaert, D. Naccache, and O. Ranen, "HOTP: An HMAC-based one-time password algorithm," IETF RFC 4226, Dec. 2005.

[21] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-based one-time password algorithm," IETF RFC 6238, May 2011.

[22] *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE 29148:2018, 2018.

[23] *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model*, ISO/IEC 25010:2023, 2023.

[24] M. Cohn, *User Stories Applied: For Agile Software Development*. Boston, MA, USA: Addison-Wesley, 2004.

[25] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.

[26] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[27] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston, MA, USA: Addison-Wesley, 2021.

[28] Vercel, Inc., *Next.js Documentation — App Router*. [Online]. Available: https://nextjs.org/docs/app

[29] NestJS, "Guards," *NestJS Documentation*. [Online]. Available: https://docs.nestjs.com/guards

[30] Prisma Data, Inc., *Prisma ORM Documentation*. [Online]. Available: https://www.prisma.io/docs

[31] R. W. Sinnott, "Virtues of the haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

[32] P. R. Winters, "Forecasting sales by exponentially weighted moving averages," *Management Science*, vol. 6, no. 3, pp. 324–342, 1960.

[33] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.

[34] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 9459–9474.

[35] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

APPENDIX A: REPOSITORY STRUCTURE

Table A.1: *Source size by area (lines, measured with wc -l on .ts/.tsx/.mjs/.cjs/.prisma/.sql files)*

Area	Path	Lines
API source	apps/api/src	7,631
Schema, migrations, seed	apps/api/prisma	2,566
API end-to-end tests	apps/api/test	600
Web pages	apps/web/app	7,013
Web components and library	apps/web/components, lib	5,734
Web browser tests	apps/web/test	146
Mobile	apps/mobile/app, src	3,245
Shared package	packages/shared/src	1,034
Total		27,969

Top-level structure of the repository as inspected (dependency and build directories omitted).

```
sportsos/
├── apps/
│   ├── api/                  NestJS 11 API
│   │   ├── prisma/           schema.prisma, migrations/ (2), seed.ts, bootstrap-admin.ts
│   │   ├── src/              achievements · ai · analytics · announcements · attendance ·
audit
│   │   │                     auth · bookings · clubs · common · complaints · facilities ·
health
│   │   │                     institution · inventory · jobs · notifications · prisma ·
rbac
│   │   │                     tasks · tournaments · users · app.module.ts · main.ts
│   │   ├── test/             smoke.mjs (83 checks) · rbac.e2e.mjs (200 checks)
│   │   └── Dockerfile
│   ├── web/                  Next.js 15 client
│   │   ├── app/(app)/        24 authenticated pages · login · register
│   │   ├── components/       shell · charts · attendance projector · qr-scanner · ui
│   │   ├── lib/              api.ts · hooks.ts · speech.ts · format.ts
│   │   ├── test/             ui-roles.e2e.cjs · ui-flows.e2e.cjs
│   │   └── Dockerfile
│   └── mobile/               Expo SDK 54 app (separate install)
│       ├── app/              (tabs)/ index · bookings · checkin · scan · profile
│       │                     book/ · booking/[id] · equipment/[code] · pass/[token] ·
login
│       └── src/              api.ts · auth.tsx · device.ts · qr.ts · hooks/
├── packages/shared/src/      roles.ts · permissions.ts · types.ts
├── docs/                     API, RBAC matrix, data integration, audits, test report
└── docker-compose.yml
```

APPENDIX B: SELECTED API DOCUMENTATION

Table B.1: *API surface by module (counted from controller decorators)*

Module	Base path	Handlers	Representative endpoints
Auth	/auth	3	POST login, POST register, GET me
Users / RBAC	/users, /rbac	4 + 10	GET/PUT/DELETE rbac/matrix, POST rbac/assignments, …/approve, …/revoke
Facilities	/facilities	6	GET :id/availability, PATCH :id/status
Bookings	/bookings	10	POST, POST check-in, POST :id/approve|reject|cancel|reschedule, GET :id/pass
Clubs	/clubs	10	POST :id/join, PATCH :id/members/:mid, POST :id/announcements
Attendance	/attendance	10	POST sessions, GET sessions/:id/otp, POST check-in, POST sessions/:id/corrections
Inventory	/inventory	23	GET items/by-qr/:code, POST issues, POST issues/:id/return, GET alerts, requests/*
Tournaments	/tournaments, /matches	13	POST :id/approve, POST :id/teams, POST :id/generate, PATCH matches/:id
Complaints	/complaints	9	PATCH :id, POST :id/escalate|close|reopen|attachments
Announcements / Tasks	/announcements, /tasks	7 + 3	POST :id/submit|approve|publish; PATCH tasks/:id/status
Notifications	/notifications	5	GET unread-count, POST read-all, POST broadcast
Analytics	/analytics	9	overview, participation, utilization, forecast, equipment, integrity, export
AI	/ai	6	GET status, POST ask, POST ask/stream (SSE), POST report
Audit / Institution / Other	various	2 + 9 + 4	GET audit/verify, POST institution/imports/:id/commit|rollback, GET health

All paths are prefixed with `/api`. "Perm." is the permission checked by the route guard; further object-level scope checks apply in the service.

Table B.2: *Selected endpoints*

Method & path	Perm.	Request (key fields)	Responses
POST /auth/login	public	email, password	200 {accessToken, user, permissions, heldRoles}; 401
GET /auth/me	any authenticated	—	200 session view
GET /facilities/:id/availability	facility.view	date (YYYY-MM-DD)	200 slot grid
POST /bookings	booking.create	facilityId, start, end, purpose, type, participants?, clubId?	201 booking (APPROVED|PENDING); 400; 409 overlap
POST /bookings/:id/approve	booking.approve	note?	200; 409 if not PENDING
POST /bookings/check-in	booking.verify	pass token (QR)	200 entry recorded; 409 too early/late
POST /attendance/sessions	attendance.manage (scoped)	title, facilityId, clubId?, endsAt	201 session (secret omitted); 403 out of scope
GET /attendance/sessions/:id/otp	attendance.manage (scoped)	—	200 {code, step, expiresInSec}

Method & path	Perm.	Request (key fields)	Responses
POST /attendance/check-in	attendance.mark	sessionId, otp, latitude, longitude, accuracyM, deviceId?	201 {outcome, message, distanceM, record}
POST /inventory/issues	inventory.issue	equipmentId | qrCode, borrower, quantity, dueAt?	201 issue; 404; 409 insufficient stock
POST /inventory/issues/:id/return	inventory.return	returnedQty, damagedQty?, note?	200 RETURNED|LOST
POST /tournaments/:id/generate	event.publish	—	200 matches (bracket or round robin)
PATCH /complaints/:id	complaint.* (role-dependent)	status?, assignedToId?, resolution?	200; 400 resolution required; 403
PUT /rbac/matrix/:role/:permission	role.manage_permissions	—	200 role permissions
POST /rbac/assignments	role.assign	userId, role, scopeType?, scopeId?, validUntil?, reason	201 ACTIVE | PENDING_APPROVAL
GET /audit/verify	audit.view	—	200 {valid, brokenAtSeq, checked, headHash}
POST /ai/ask/stream	ai.use	question	SSE: sources, delta…, done

APPENDIX C: DATABASE ENTITY CATALOGUE

Table C.1: *All 31 Prisma models grouped by domain*

Domain	Models
Identity & access	User, RoleAssignment, RolePermission, PermissionDefinition
Facilities & bookings	Facility, Booking
Clubs	Club, ClubMembership
Verified attendance	AttendanceSession, AttendanceRecord, AttendanceAttempt, AttendanceCorrection
Inventory	Equipment, EquipmentIssue, InventoryTransaction, EquipmentRequest
Tournaments	Tournament, Team, TeamMember, Match
Engagement	Notification, Complaint, ComplaintAttachment, Announcement, OperationalTask, Achievement
Governance & data	AuditLog, AiUsageLog, InstitutionalRecord, ImportBatch, ImportBatchItem

Table C.2: *Seeded synthetic data volumes (run of 12 Nov 2026)*

Entity	Rows	Entity	Rows
User	79	AttendanceSession	120
RoleAssignment	156	AttendanceRecord	1,080
Facility	9	Equipment	15
Club	5	EquipmentIssue	283
Booking	2,798	Tournament	3
Complaint	5	Notification	83
RolePermission	346	AuditLog	588 (589 after a test action)

APPENDIX D: IMPLEMENTATION EVIDENCE MATRIX

Each requirement is traced to the code that implements it and the test that verified it on 12 November 2026.

Table D.1: *Requirement → implementation → verification evidence*

Req.	Implementation (file)	Verification evidence	Status
FR-01	auth/auth.service.ts, auth.controller.ts	AUTH-01…08; smoke "login", "register"	Verified
FR-02	— (no route)	AUTH-09 → 404	Not implemented
FR-03	rbac/role-assignments.service.ts	RBAC-10…13; e2e "Role assignment safeguards" (24)	Verified
FR-04	rbac/rbac.service.ts (setGrant, resetRole)	RBAC-14; e2e matrix grant/revoke; UI flow	Verified
FR-05	facilities/facilities.service.ts	smoke "availability grid"; e2e scoped ground staff	Verified
FR-06	bookings/bookings.service.ts	BK-01…06	Verified
FR-07	bookings (check-in, pass), common/qr.ts	BK-07, BK-08; smoke "gate pass QR"	Verified
FR-08	attendance/attendance.service.ts, common/otp.ts, common/geo.ts	ATT-01…11; otp/geo unit tests	Verified
FR-09	attendance (corrections)	ATT-12; e2e "Attendance corrections" (7)	Verified
FR-10	inventory/inventory.service.ts (applyStock)	INV-01…06	Verified
FR-11	inventory alerts; jobs/jobs.service.ts	smoke "alerts: 2 low, 0 overdue"	Verified (alerts); job timing not tested
FR-12	tournaments/*.ts, bracket.ts	TRN-01, TRN-02; bracket unit tests; RBAC-08	Verified
FR-13	complaints/complaints.service.ts	CMP-01…05; e2e "Complaints" (16)	Verified
FR-14	notifications, announcements modules	smoke "notifications", "broadcast"; e2e announcements	Verified
FR-15	analytics/analytics.service.ts, forecast.ts	smoke analytics/* (6); forecast unit tests	Verified (synthetic data)
FR-16	ai/ai.service.ts, knowledge.service.ts, vector-store.ts	smoke AI (5); e2e AI isolation (5); unit (8, mocked LLM)	Partial
FR-17	apps/mobile/*	tsc --noEmit exit 0	Partial
FR-18	audit/audit.service.ts, common/hash-chain.ts	DB-04; hash-chain unit tests	Verified
NFR-03	advisory locks; conditional UPDATE	BK-04, INV-04 (small-scale concurrency)	Partly verified
NFR-07	docker-compose.yml, Dockerfiles	not executed	Configured
NFR-08	—	not tested	Not tested

APPENDIX E: ADDITIONAL SCREENSHOTS

All screenshots were captured from the production web build on 12 November 2026 using synthetic demo data.

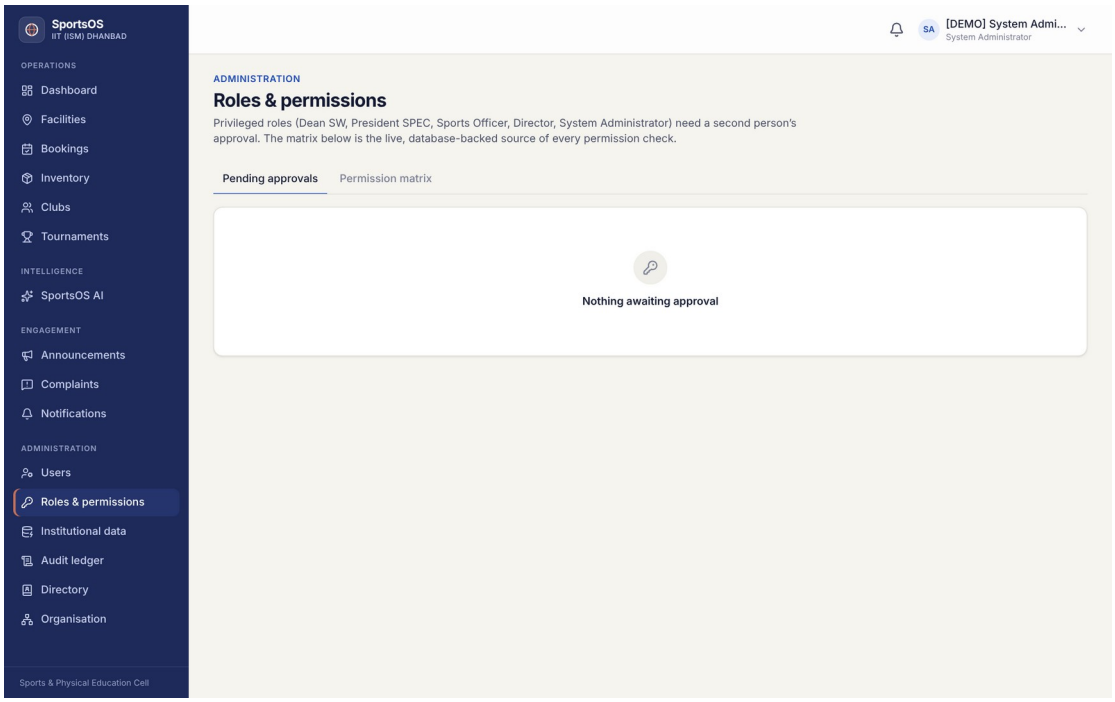


Figure E.1: *Administrator view of roles and the editable role × permission matrix (synthetic data)*

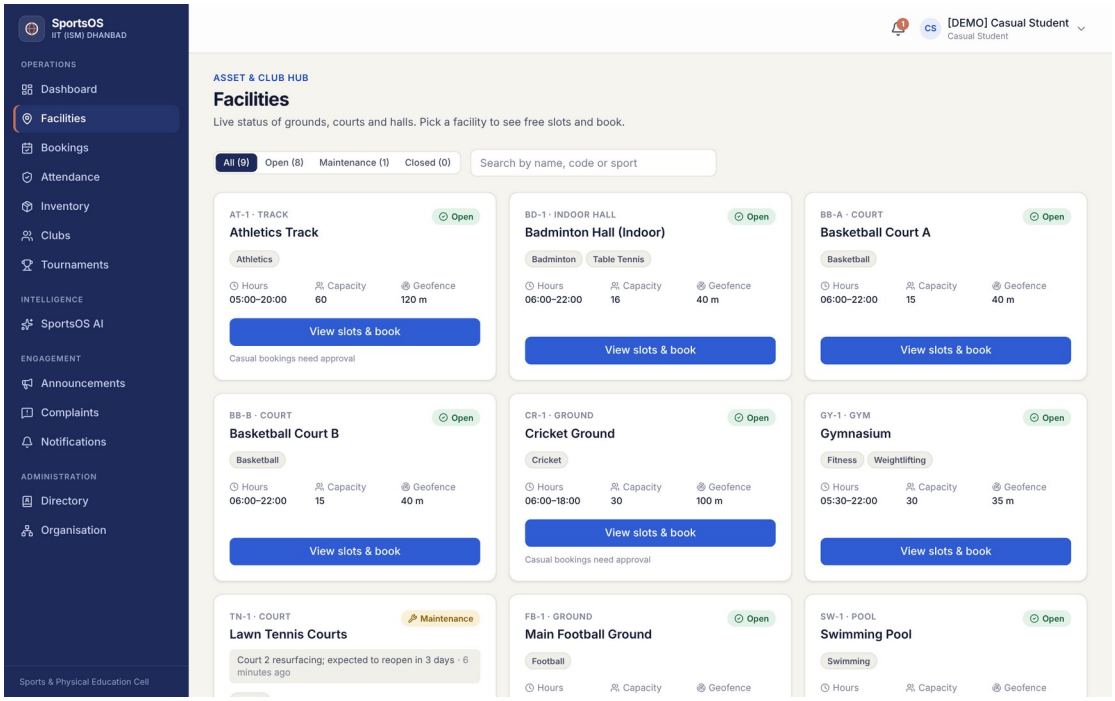


Figure E.2: *Facility page with the daily availability grid (Main Football Ground) (synthetic data)*

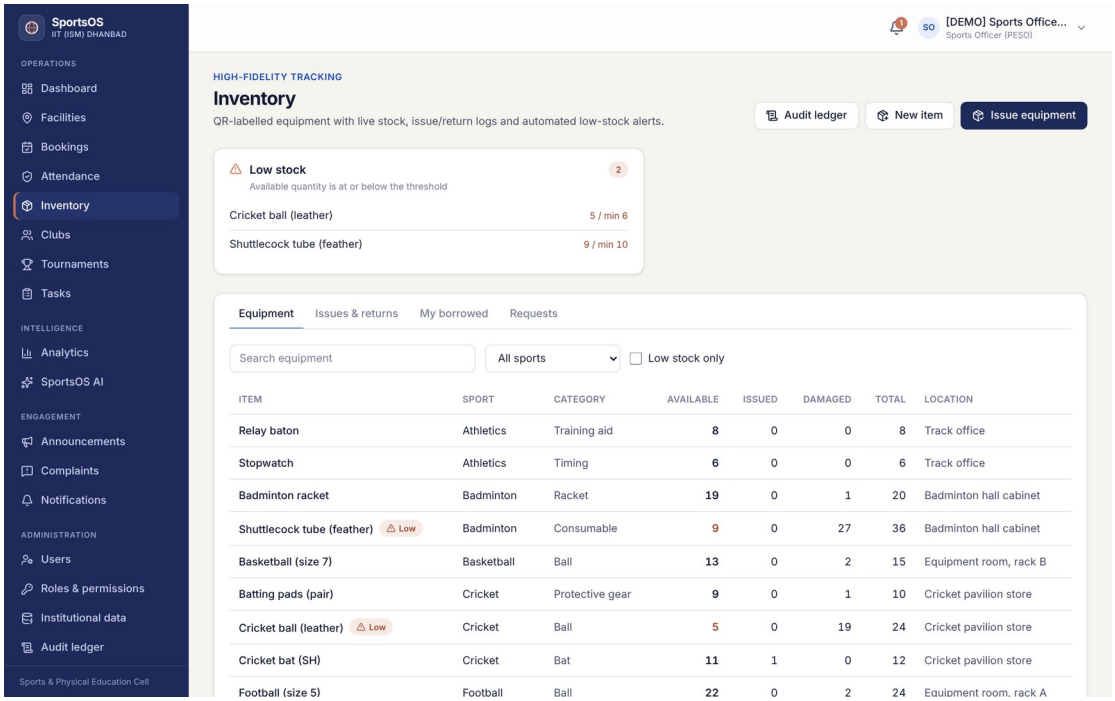


Figure E.3: *Inventory view with stock levels and low-stock alerts (synthetic data)*

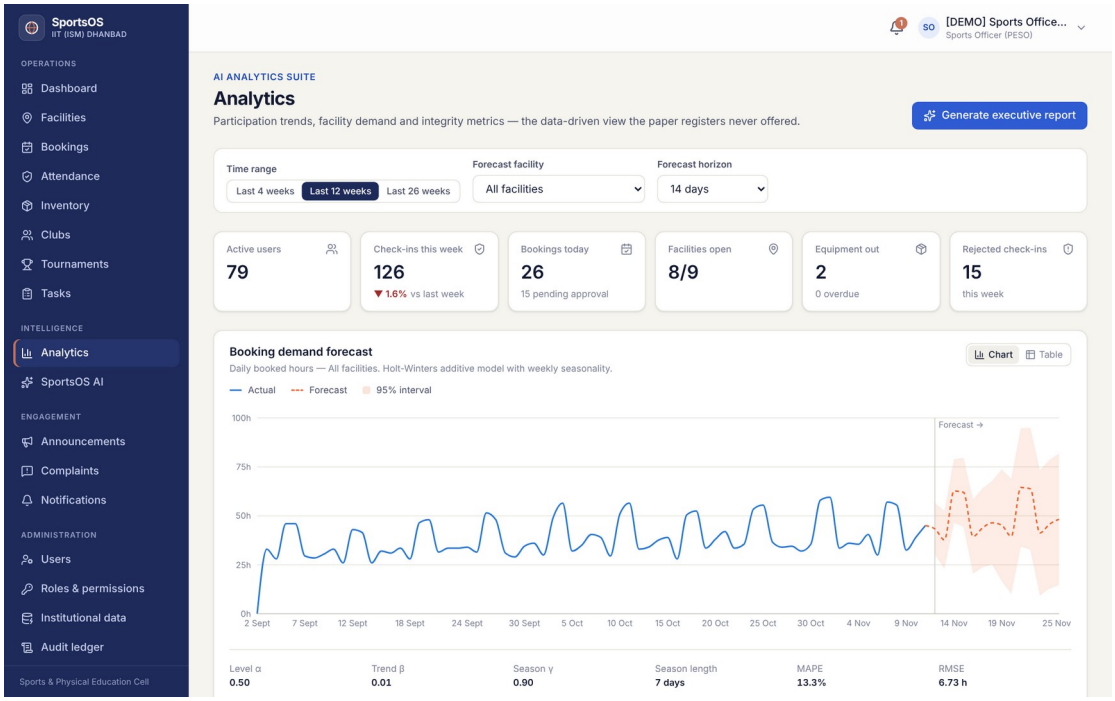


Figure E.4: *Analytics page: participation, utilisation and forecast views (synthetic data)*

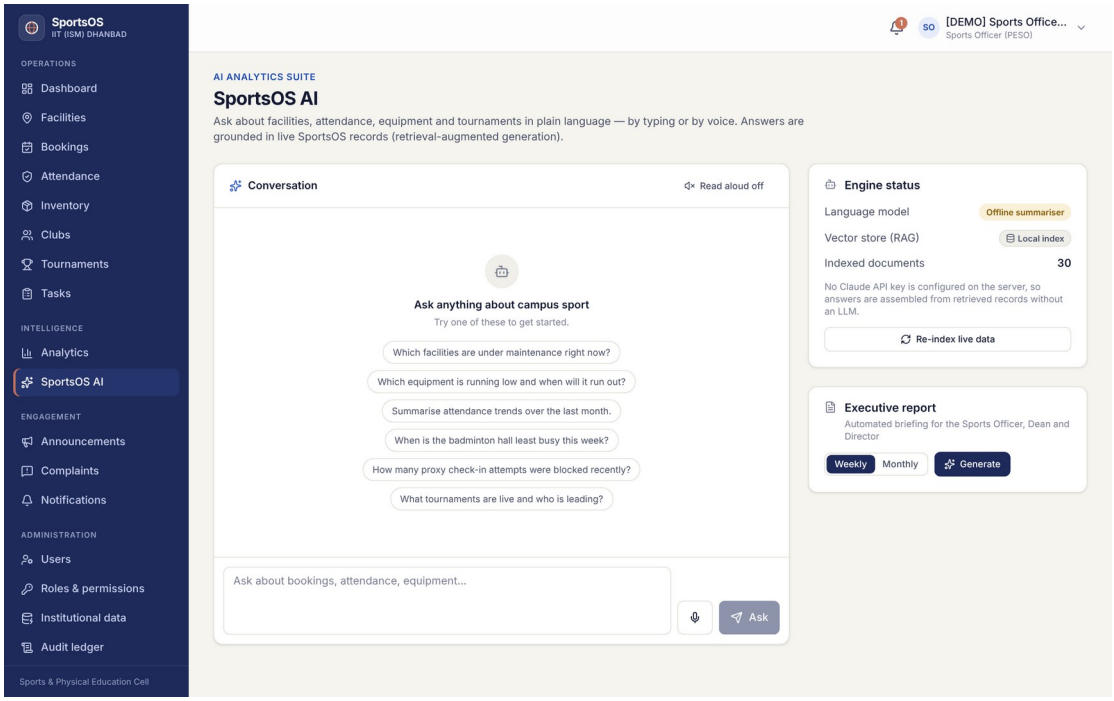


Figure E.5: *SportsOS AI assistant page (offline retrieval mode) (synthetic data)*

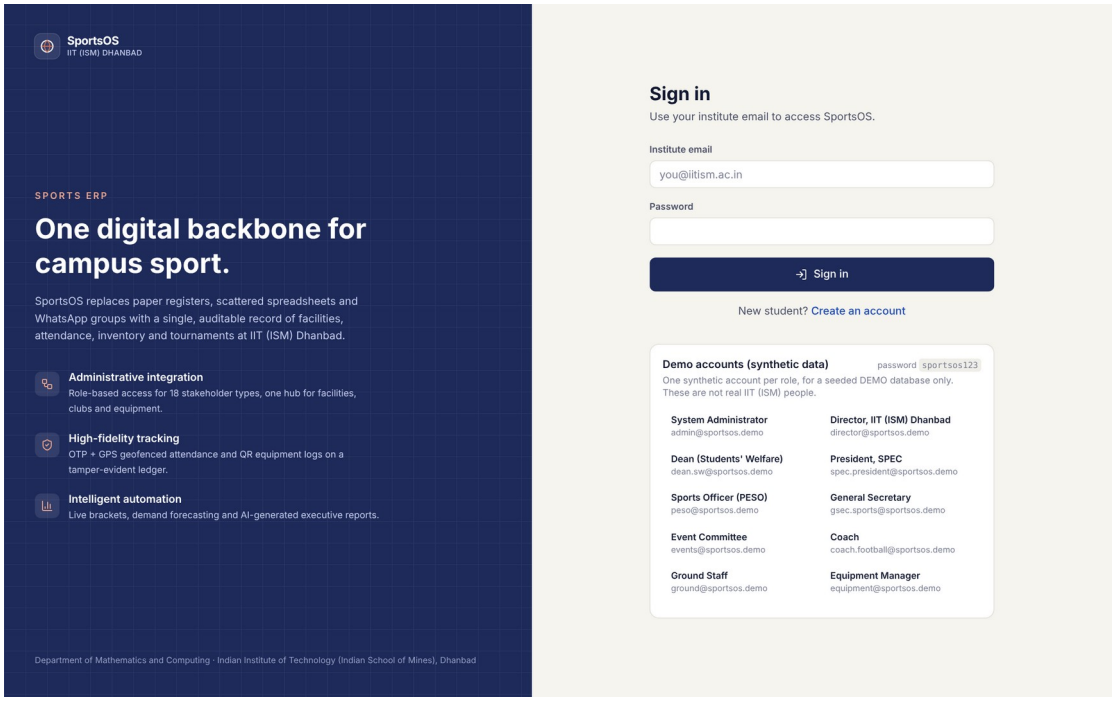


Figure E.6: *Sign-in page (synthetic data)*

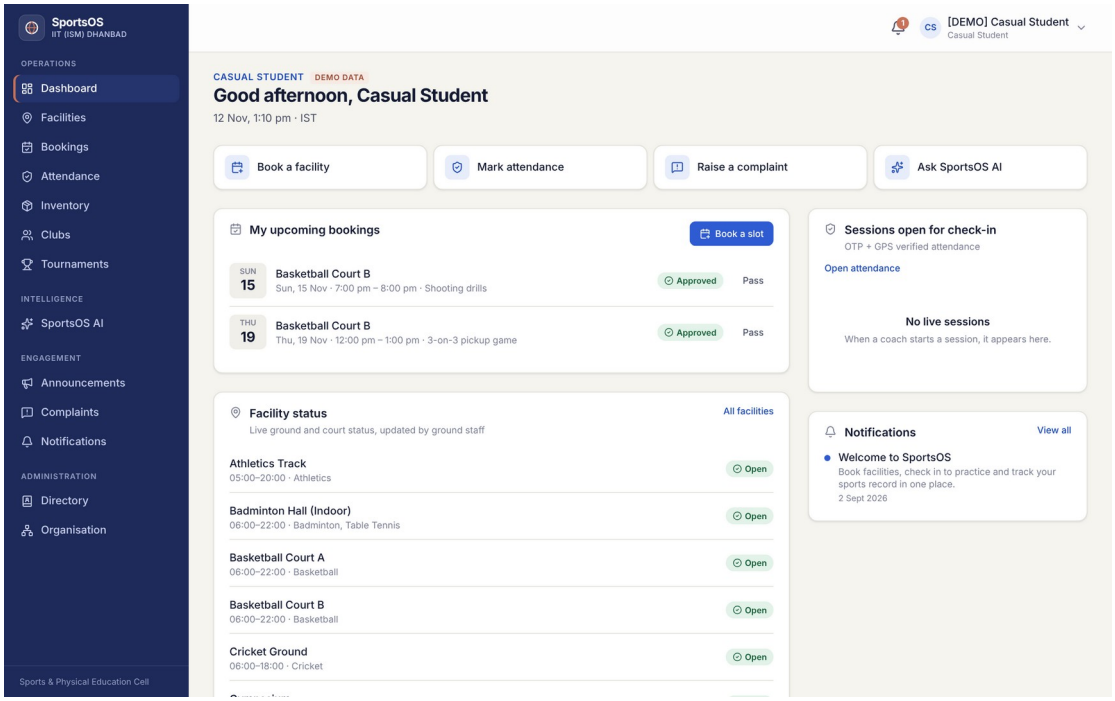


Figure E.7: *Student dashboard (synthetic data)*

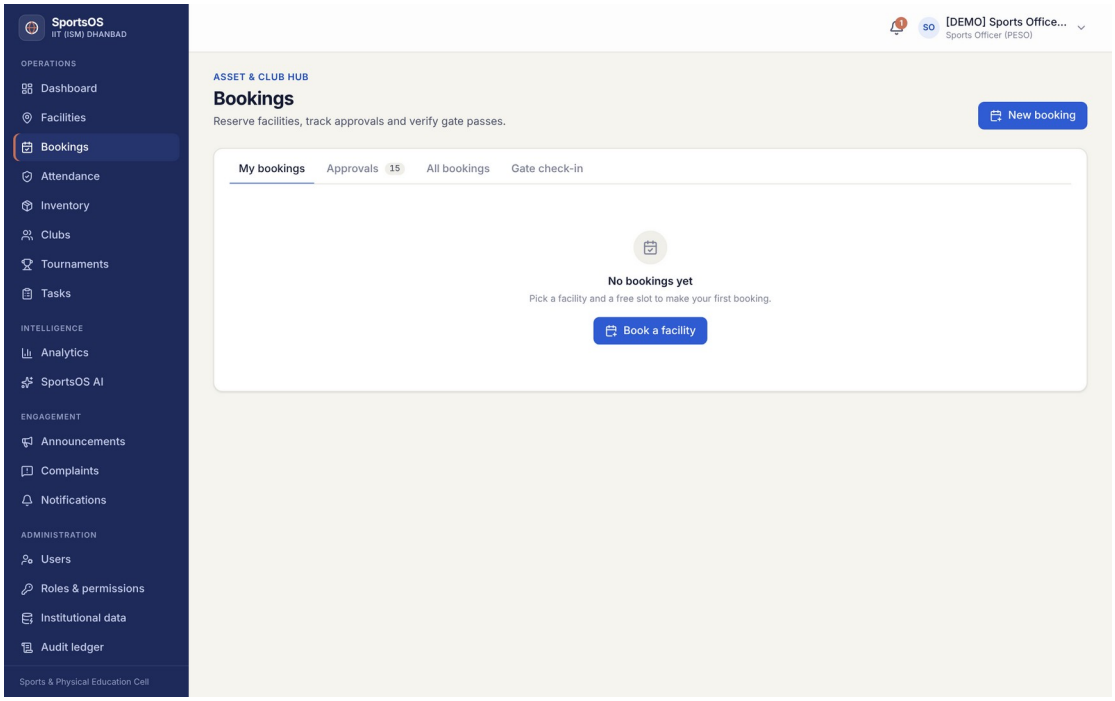


Figure E.8: *Booking approvals for the Sports Officer (synthetic data)*

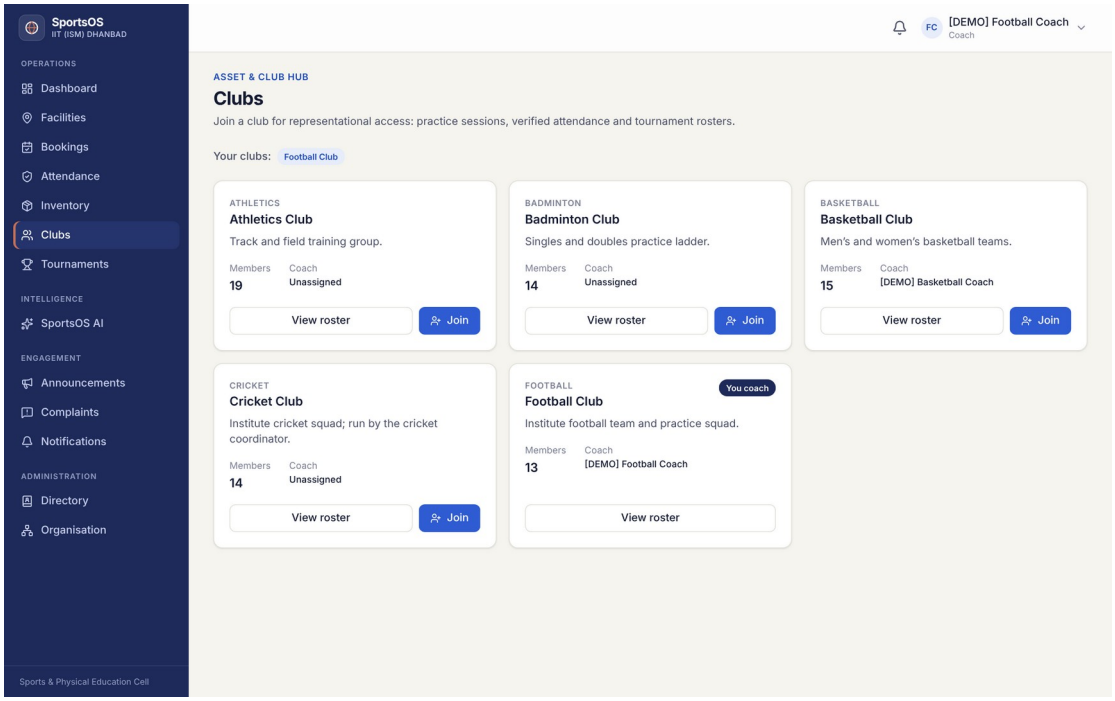


Figure E.9: *Clubs as seen by a coach (synthetic data)*

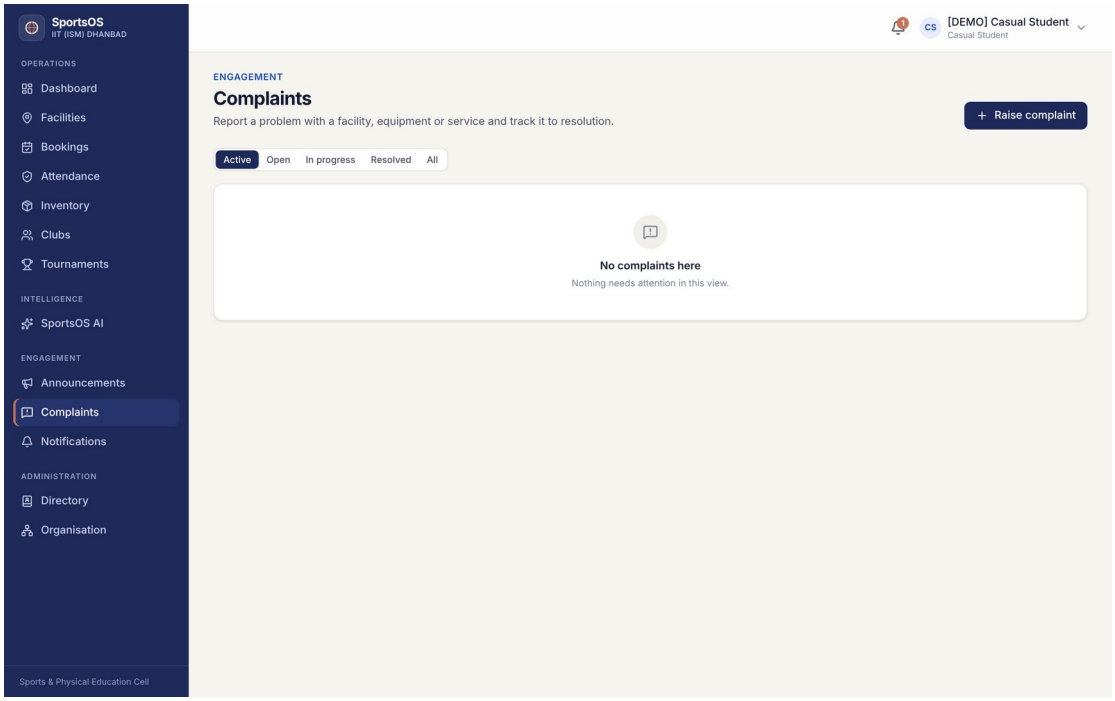


Figure E.10: *Complaints page for a student (synthetic data)*

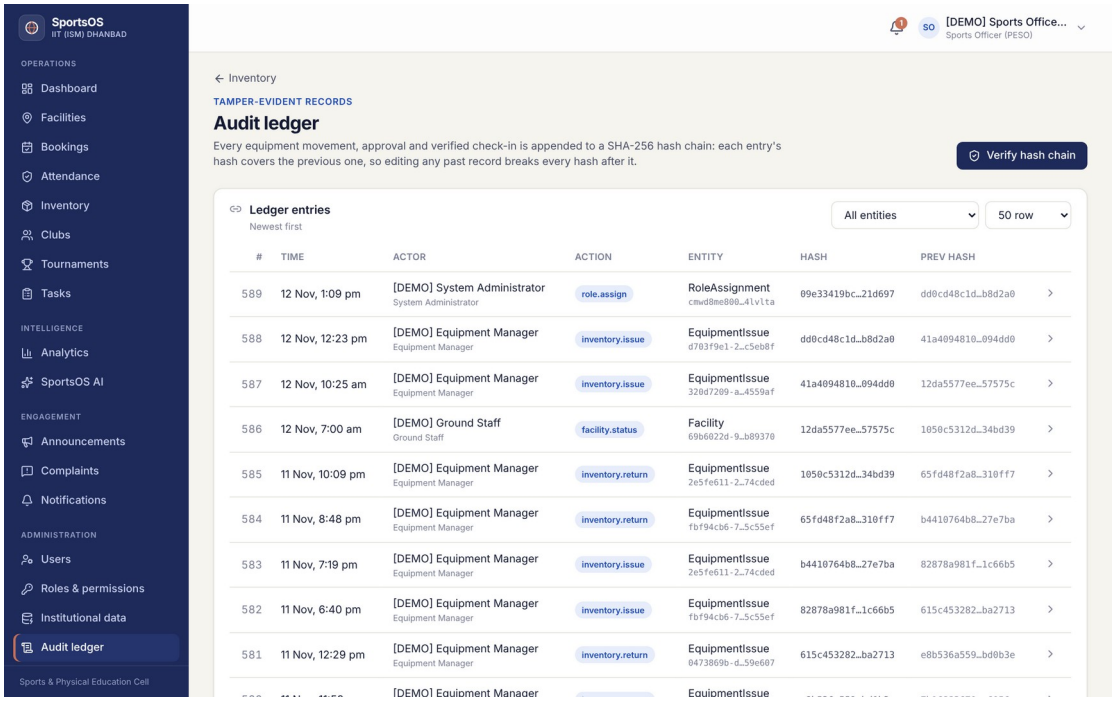


Figure E.11: *Hash-chained audit ledger view (synthetic data)*

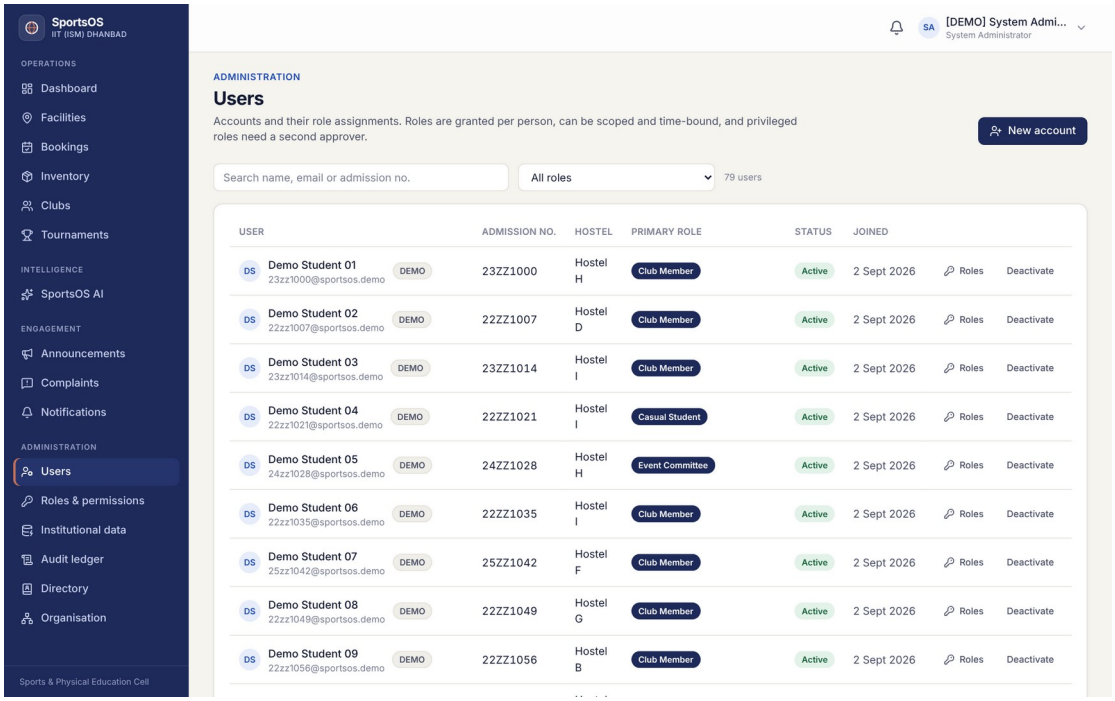


Figure E.12: *User administration (synthetic data)*

APPENDIX F: DETAILED TEST CASES

Complete test-case tables from the verification run of 12 November 2026. BK, INV, CMP and TRN cases come from the API smoke and RBAC/workflow e2e suites; AUTH-01, AUTH-09, ATT-01 to ATT-11 and the DB cases were executed directly by the author.

Table F.1: *Functional test cases: booking, inventory and complaints (results from the smoke and e2e runs)*

Test ID	Test scenario	Expected result	Actual result	Status
BK-01	Student books a free slot on a facility that needs no approval	Booking APPROVED	APPROVED	PASS
BK-02	Overlapping booking for the same facility	HTTP 409	409 "Slot overlaps an existing booking"	PASS
BK-03	Booking outside opening hours	HTTP 400	400 "open 06:00–22:00 IST"	PASS
BK-04	6 concurrent requests for one slot	Exactly one succeeds	201 ×1, 409 ×5	PASS
BK-05	Booking on a facility that needs approval	PENDING; officer can approve	PENDING → APPROVED	PASS
BK-06	Approving an already approved booking	HTTP 409	409	PASS
BK-07	Gate check-in more than 15 min early	HTTP 409	409 "Too early"	PASS
BK-08	Student opens another student's pass	HTTP 404	404	PASS
INV-01	Issue item by QR to a borrower identified by admission no.	Issue created, stock −1	Created; stock decremented	PASS
INV-02	Issue more than available	HTTP 409	409 "Only 11 × Basketball (size 7) available"	PASS
INV-03	Return with damaged quantity	available/damaged updated	available 13 → 12, damaged +1	PASS
INV-04	8 concurrent issues on 3 units	Exactly 3 succeed	201 ×3, 409 ×5	PASS
INV-05	Ledger deltas vs stock after all movements	Reconcile exactly	Reconciled (10 txns)	PASS
INV-06	Requester approves own equipment request	Refused	Refused	PASS
CMP-01	Resolve complaint without a note	HTTP 400	400	PASS
CMP-02	Assignee resolves; raiser reopens	RESOLVED → OPEN	As expected	PASS
CMP-03	Raiser tries to escalate	Refused	Refused	PASS
CMP-04	Officer escalates; Dean views queue	Visible to Dean	Visible	PASS
CMP-05	Upload script disguised as an image	Rejected	Rejected by content sniffing	PASS
TRN-01	Generate knock-out bracket; play to champion	Byes, advancement, champion	7 matches; champion set	PASS
TRN-02	Event Committee approves its own proposal	Refused	Refused	PASS

Table F.2: *Authentication test cases*

Test ID	Test scenario	Expected result	Actual result	Status
AUTH-01	Sign in with each of the 19 seeded role accounts	HTTP 200 + token for all	19 / 19 returned 200 with role and permissions	PASS
AUTH-02	Sign in with an unknown account	HTTP 401	401	PASS
AUTH-03	Login response contains the password hash	Never	Absent	PASS

Test ID	Test scenario	Expected result	Actual result	Status
AUTH-04	Protected route without a token	HTTP 401	401	PASS
AUTH-05	Token with a tampered signature	HTTP 401	401	PASS
AUTH-06	Register with a non-institute email	HTTP 400	400	PASS
AUTH-07	Register with an institute email	Account with STUDENT role	STUDENT	PASS
AUTH-08	Use a token after the account is deactivated	Rejected	Rejected	PASS
AUTH-09	POST /auth/forgot-password	Reset flow	404: endpoint does not exist	Not impl.

Table F.3: *Authorisation and RBAC test cases*

Test ID	Test scenario	Expected result	Actual result	Status
RBAC-01	Student calls GET /analytics/overview	HTTP 403	403	PASS
RBAC-02	Student sends X-Active-Role: SPORTS_OFFICER	HTTP 403 (not ignored)	403	PASS
RBAC-03	Unknown body field (mass assignment)	HTTP 400	400	PASS
RBAC-04	Football coach opens a session for the Basketball club	HTTP 403	403 (author's check ATT-02)	PASS
RBAC-05	Football coach opens a session for the Football club	HTTP 201	201 (ATT-01)	PASS
RBAC-06	Basketball coach reads the Football session OTP	Refused	403 (ATT-03)	PASS
RBAC-07	Coach edits another club's roster / reads its report	Refused	Refused	PASS
RBAC-08	Hostel A secretary registers a team for another hostel	Refused	Refused	PASS
RBAC-09	Facility-scoped ground staff changes another facility	Refused	Refused	PASS
RBAC-10	Sports Officer or Dean grants ADMIN	Refused	Refused	PASS
RBAC-11	Privileged role assigned → effective?	PENDING until a second person approves	PENDING → approved by admin → effective	PASS
RBAC-12	Revoke a role; reuse the same token	Revocation immediate	Immediate	PASS
RBAC-13	Temporary role past validUntil (before expiry job runs)	No longer effective	Not effective	PASS
RBAC-14	Remove matrix-management permission from ADMIN	Refused (lock-out protection)	Refused	PASS
RBAC-15	Per-role navigation in the browser (9 roles)	Expected menu items only	9 / 9 as expected; 0 console errors	PASS

Table F.4: *Database test cases (author's checks)*

Test ID	Test scenario	Expected result	Actual result	Status
DB-01	Validate schema; apply migrations to an empty database	Valid; migrations applied	Valid; 2 migrations applied	PASS
DB-02	Permission bootstrap on first API start	70 definitions, 346 default grants	PermissionDefinition = 70; RolePermission = 346	PASS
DB-03	Persistence across API restart	Row counts unchanged	Bookings 2,798 → 2,798; AuditLog 589 → 589	PASS
DB-04	Audit chain verification (GET /audit/verify)	valid = true	valid; 589 entries checked; brokenAtSeq = null	PASS

Test ID	Test scenario	Expected result	Actual result	Status
DB-05	Duplicate attendance record for one student	Prevented by unique index	DUPLICATE (ATT-08)	PASS
DB-06	Stock invariant after concurrent issues	total = available + issued + damaged	Holds	PASS

Table F.5: *Attendance verification test cases (author's run against a live session)*

Test ID	Test scenario	Expected result	Actual result	Status
ATT-01	Coach opens a session for own club	HTTP 201	201	PASS
ATT-02	Coach opens a session for a club they do not coach	HTTP 403	403	PASS
ATT-03	Other coach reads the session OTP	Refused	403	PASS
ATT-04	Member submits a wrong OTP	BAD_OTP	BAD_OTP	PASS
ATT-05	Correct OTP, position ≈ 1.1 km away	OUT_OF_GEOFENCE	OUT_OF_GEOFENCE, 1,112 m	PASS
ATT-06	Correct OTP, GPS accuracy 250 m	LOW_ACCURACY	LOW_ACCURACY	PASS
ATT-07	Member, correct OTP, inside geofence	ACCEPTED	ACCEPTED, 11 m from centre	PASS
ATT-08	Same member checks in again	DUPLICATE	DUPLICATE	PASS
ATT-09	Non-member checks in to a club session	NOT_ELIGIBLE	NOT_ELIGIBLE	PASS
ATT-10	Second student uses the first student's device (smoke test)	DEVICE_REUSED + coach alert	DEVICE_REUSED	PASS
ATT-11	Non-numeric OTP	HTTP 400	400	PASS
ATT-12	Coach adds a member manually	Record MANUAL, no coordinates	MANUAL, latitude = null	PASS